

CHRIST (DEEMED TO BE UNIVERSITY)

Department of Computer Science

FINAL COMPREHENSIVE PROJECT REPORT

A Data-Centric Platform for Investment Analytics and Financial Market Decision Support

Project Name: **FinHQ**

Name : Abhijith E
Register Number : 2448503
Email : abhijith.e@msam.christuniversity.in
Repository : <https://github.com/Abhijith-E/fintechphase2>
Date : March 28, 2026

Academic Year 2025–2026

CERTIFICATE

This is to certify that the final comprehensive project report entitled **A Data-Centric Platform for Investment Analytics and Financial Market Decision Support** is a bonafide record of the work carried out by **Abhijith E** bearing Register Number **2448503**, Department of Computer Science, CHRIST (Deemed to be University), during the academic year 2025–2026.

This report is based on the implemented project available in the repository <https://github.com/Abhijith-E/fintechphase2> and presents the expanded technical analysis, design documentation, implementation discussion, and testing-oriented study of the system.

Guide

Project Coordinator

Head of the Department

External Examiner

Place: Bengaluru

Date: March 28, 2026

ACKNOWLEDGMENTS

I express my sincere gratitude to my guide for the academic direction, technical review, and constructive suggestions that helped shape this final comprehensive report. The guidance received during the course of this work contributed significantly to improving the clarity of system design, architectural explanation, module-level interpretation, and the overall professionalism of the documentation.

I also thank the Department of Computer Science, CHRIST (Deemed to be University), for creating an academic environment that encourages both software implementation and disciplined documentation. The departmental reporting standards have influenced the structure, completeness, and presentation quality of this document.

I acknowledge all those who supported the project through discussion, review, feedback, testing ideas, and encouragement. Their contributions helped improve both the technical understanding of the system and the presentation of its design, implementation, and future scope.

ABSTRACT

This final report presents FinHQ, a data-centric platform designed for investment analytics and financial market decision support. The project integrates secure authentication, stock discovery, technical analysis, fundamental research, market news intelligence, paper trading, strategies, alerts, education, community interaction, and machine-learning-assisted analytics inside a unified architecture. The system is implemented using a Next.js frontend, a FastAPI backend, a dedicated FastAPI ML microservice, PostgreSQL-oriented persistence, Redis support, and Docker-based orchestration. The scope of this final report expands beyond both the draft and revised versions by documenting the problem definition, requirements, conceptual models, architecture, data model, module structure, API contracts, implementation strategy, testing approach, operational considerations, AI methodology, security posture, and future roadmap in greater detail. The project demonstrates secure account handling, modular service decomposition, chart-based market analysis, risk and forecasting support, and a strong data model that can support future growth. The report concludes that FinHQ is a technically credible and extensible platform for educational and prototype fintech use, while also identifying areas such as live integration, automated testing, operational hardening, and product-scale refinement for future enhancement.

CONTENTS

Acknowledgments	iii
Abstract	iv
List of Tables	xii
List of Figures	xiii
Abbreviations	xiv
1 INTRODUCTION	1
1.1 PROJECT DESCRIPTION	1
1.2 EXISTING SYSTEM	2
1.3 OBJECTIVES	2
1.4 PURPOSE, SCOPE AND APPLICABILITY	3
1.4.1 Purpose	3
1.4.2 Scope	3
1.4.3 Applicability	4
1.5 OVERVIEW OF THE REPORT	4
2 SYSTEM ANALYSIS AND REQUIREMENTS	5
2.1 PROBLEM DEFINITION	5
2.2 REQUIREMENTS SPECIFICATION	5
2.2.1 Functional Requirements	6
2.2.2 Non-Functional Requirements	6
2.3 BLOCK DIAGRAM	7
2.4 SYSTEM REQUIREMENTS	7
2.4.1 User Characteristics	7
2.4.2 Software and Hardware Requirements	8

2.4.3	Constraints	8
2.5	CONCEPTUAL MODELS	9
2.5.1	Data Flow Diagram	9
2.5.2	ER Diagram	9
3	SYSTEM DESIGN	10
3.1	SYSTEM ARCHITECTURE	10
3.2	MODULE DESIGN	10
3.2.1	Authentication and Identity Module	11
3.2.2	Dashboard and Portfolio Module	11
3.2.3	Technical Analysis Module	11
3.2.4	Fundamental Analysis and News Module	11
3.2.5	Trading, Alerts, and Strategy Module	11
3.2.6	Community and Learning Module	11
3.2.7	ML and Quantitative Module	12
3.3	DATABASE DESIGN	12
3.3.1	Tables and Relationships	12
3.3.2	Data Integrity and Constraints	13
3.4	SYSTEM CONFIGURATION	13
3.5	INTERFACE AND PROCEDURAL DESIGN	14
3.5.1	User Interface Design	14
3.5.2	Application Flow / Class Diagram	14
3.6	REPORTS DESIGN	15
4	IMPLEMENTATION	16
4.1	IMPLEMENTATION APPROACHES	16
4.2	CODING STANDARD	16
4.3	CODING DETAILS	17
4.3.1	Authentication and Security Implementation	17
4.3.2	Market Data Implementation	17

4.3.3	Trading and Alert Implementation	17
4.3.4	ML Implementation	17
4.4	SCREEN SHOTS	17
5	TESTING	18
5.1	TEST CASES	19
5.2	TESTING APPROACHES	20
5.2.1	Unit Testing	20
5.2.2	Integration Testing	20
5.2.3	System and Workflow Testing	20
5.3	TEST REPORTS	21
6	CONCLUSION	22
6.1	DESIGN AND IMPLEMENTATION ISSUES	22
6.2	ADVANTAGES AND LIMITATIONS	22
6.2.1	Advantages	22
6.2.2	Limitations	23
6.3	FUTURE SCOPE OF THE PROJECT	23
7	DETAILED MODULE AND FUNCTIONAL CATALOG	24
7.1	FRONTEND MODULE CATALOG	24
7.1.1	Authentication Routes	24
7.1.2	Dashboard and Portfolio Routes	24
7.1.3	Analysis Routes	25
7.1.4	Action and Simulation Routes	25
7.1.5	Engagement and Learning Routes	25
7.2	BACKEND FUNCTIONAL CATALOG	25
7.2.1	Authentication and User Management	25
7.2.2	Market and Research Services	26
7.2.3	Trading and Alert Services	26
7.2.4	Strategies, Education, and Social Services	26

7.3	ML FUNCTIONAL CATALOG	26
7.3.1	Feature Engineering	26
7.3.2	Forecasting	26
7.3.3	Risk Analytics	27
7.3.4	Recommendation and Sentiment	27
7.3.5	Backtesting and Pattern Detection	27
8	DATA-CENTRIC ARCHITECTURE AND DATA MANAGEMENT	28
8.1	WHY THE PLATFORM IS DATA-CENTRIC	28
8.2	PRIMARY DATA DOMAINS	28
8.3	DATA FLOW ACROSS LAYERS	29
8.4	DATA QUALITY CONSIDERATIONS	29
8.5	DATA SECURITY AND PRIVACY	29
9	API CONTRACTS, ROUTES, AND INTEGRATION SURFACES	30
9.1	BACKEND ROUTE FAMILIES	30
9.1.1	Authentication Family	30
9.1.2	User Family	30
9.1.3	Stocks Family	30
9.1.4	Trading Family	30
9.1.5	Alerts, Strategy, Social, and Education Families	30
9.2	ML ROUTE FAMILIES	31
9.3	API DESIGN STRENGTHS	31
9.4	INTEGRATION CHALLENGES	31
10	AI, QUANTITATIVE METHODS, AND MODELING STRATEGY	32
10.1	ROLE OF AI IN THE PLATFORM	32
10.2	TIME-SERIES PREDICTION	32
10.3	RISK ANALYSIS	32
10.4	PATTERN DETECTION	32
10.5	FUNDAMENTAL AND SENTIMENT ANALYSIS	33

10.6	MODEL GOVERNANCE AND FUTURE NEEDS	33
11	DEPLOYMENT, OPERATIONS, AND ENGINEERING MANAGEMENT	34
11.1	CONTAINERIZED DEPLOYMENT	34
11.2	SERVICE RESPONSIBILITIES	34
11.3	OPERATIONS AND MAINTENANCE	34
11.4	PROJECT MANAGEMENT VIEW	34
11.5	QUALITY RISKS	35
12	COMPREHENSIVE EVALUATION	36
12.1	ACADEMIC VALUE	36
12.2	TECHNICAL VALUE	36
12.3	PRODUCT VALUE	36
12.4	LIMITATION SUMMARY	37
12.5	FINAL EVALUATION	37
A	USER’S MANUAL	38
A.1	Installation and Setup	38
A.2	Basic Usage Flow	38
A.3	Maintenance Guidance	38
B	ADDITIONAL SUPPORTING TABLES	39
B.1	Frontend Route Inventory	39
B.2	Backend Service Inventory	39
B.3	ML Service Inventory	40
C	PLANNING AND DESIGN CONCEPTS	41
C.1	Indicative Gantt-like Plan	41
C.2	Design Concepts Consideration	41
D	EXTENDED FRONTEND AND SERVICE ANALYSIS	42
D.1	Frontend Screen Analysis: Login	42

D.2	Frontend Screen Analysis: Register	42
D.3	Frontend Screen Analysis: Forgot Password and Reset Password	42
D.4	Frontend Screen Analysis: Dashboard	42
D.5	Frontend Screen Analysis: Portfolio	43
D.6	Frontend Screen Analysis: Technical and Fundamental Research	43
D.7	Frontend Screen Analysis: News, Risk, Backtest, Trade, Alerts, Community, and Learn	43
D.8	Backend Service Analysis: StockDataService	43
D.9	Backend Service Analysis: BrokerService	43
D.10	Backend Service Analysis: AlertService, EducationService, and SocialService	44
D.11	Backend Service Analysis: WebSocket Connection Manager	44
D.12	ML Service Analysis	44
E	EXTENDED API CONTRACT AND ENDPOINT ANALYSIS	45
E.1	Authentication and User Endpoints	45
E.2	Stock and Research Endpoints	45
E.3	Trading and Alert Endpoints	45
E.4	Strategy, Social, and Education Endpoints	45
E.5	ML Endpoints	46
E.6	Endpoint-Level Engineering Significance	46
F	EXTENDED DATA MODEL, SECURITY, AND REQUIREMENT TRACEABILITY	47
F.1	Data Model Deep Dive	47
F.2	Security Control Review	47
F.3	Functional Requirement Traceability	47
F.4	Non-Functional Traceability	48
G	EXTENDED OPERATIONAL AND SUBMISSION NOTES	49
G.1	User Manual Perspective	49

G.2 Submission Perspective	49
G.3 Final Note on Report Completeness	49
H BACKEND API INVENTORY	50
I ML API INVENTORY	52
J DATA DICTIONARY AND ENTITY REGISTER	53
K SECURITY CONTROL REGISTER	55
L USE CASE AND REQUIREMENT TRACEABILITY MATRIX	56
M FORMAL TEST CASE MATRIX	57
N MODULE-TO-SCREEN AND SERVICE MAPPING	60
O IMPLEMENTATION MILESTONE SUMMARY	62
P OPERATIONAL RISK AND MITIGATION REGISTER	64
References	66

LIST OF TABLES

2.1	User characteristics	7
2.2	Software requirements	8
2.3	Indicative hardware requirements	8
3.1	Key tables and relationships	12
5.1	Expanded representative test cases	19
5.2	Revised test report view	21
C.1	Indicative phase-wise project progression	41

LIST OF FIGURES

2.1	High-level block diagram of FinHQ	7
2.2	Conceptual data flow of the system	9
2.3	Conceptual entity relationship view	9
3.1	System architecture view	10
3.2	Conceptual UI navigation structure	14
3.3	Application flow of the platform	14

ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
AI	Artificial Intelligence
ML	Machine Learning
UI	User Interface
UX	User Experience
JWT	JSON Web Token
TOTP	Time-based One-Time Password
OHLCV	Open, High, Low, Close, Volume
DFD	Data Flow Diagram
ER	Entity Relationship
ORM	Object Relational Mapping
P&L	Profit and Loss
VaR	Value at Risk
MACD	Moving Average Convergence Divergence
RSI	Relative Strength Index
LSTM	Long Short-Term Memory
CNN	Convolutional Neural Network
CI	Continuous Integration
DB	Database

CHAPTER 1

INTRODUCTION

This chapter presents the background of the project, the need for the system, and the way the report is organized.

1.1 PROJECT DESCRIPTION

FinHQ is a software platform intended to support investment analytics and financial market decision support through a structured, data-centric system design. The project combines multiple workflows that are often scattered across independent applications: secure user access, stock discovery, technical analysis, fundamental evaluation, market news tracking, paper-trading simulation, risk measurement, chart-pattern analytics, and learning-oriented guidance.

The system is built around the idea that financial decision support is primarily a data problem rather than only a user-interface problem. Investors and analysts do not simply consume one type of information; they continuously move between historical price data, technical indicators, company fundamentals, market-moving events, account state, order status, and interpretive intelligence. Therefore, the project treats data collection, transformation, persistence, and presentation as first-class concerns.

The repository implementation shows that the project is not a narrow demonstration. It includes a Next.js frontend for route-driven interaction [2], [3], a FastAPI backend for transactional business logic [4], a separate FastAPI ML microservice for analytics and model-oriented computation [4], PyTorch-backed model workflows [7], and Docker-based orchestration for deployment and reproducibility [5]. This combination makes the project academically significant because it demonstrates integration of multiple layers of modern software engineering rather than isolated experimentation.

Another important aspect of the project is its focus on practical usability. The route structure maps directly to user tasks such as logging in, viewing a dashboard, exploring technical analysis, reading asset-related news, placing a simulated trade, creating an alert, or inspecting risk. This task-oriented design improves clarity and lowers the cognitive burden on the user.

1.2 EXISTING SYSTEM

The existing system landscape in the broader market can be described as fragmented, partially integrated, and frequently inconsistent in depth. A user may rely on one application for price charts, another for company fundamentals, another for alerts, and another for news. Even when applications provide multiple features, those features are often weakly connected and do not form a coherent research-to-decision workflow.

In academic and prototype contexts, the situation is often more limited. Many student projects focus only on prediction or only on charting, without secure identity management, persistent domain modeling, or realistic modular architecture. Some projects emphasize visuals while ignoring backend design. Others expose a few APIs but lack a serious data model or report-worthy explanation of architecture.

The existing system problem is therefore not just functional incompleteness. It is also architectural incompleteness. A credible investment-support platform must combine authentication, protected resources, domain-specific persistence, analytical services, workflow continuity, and room for future expansion. Without these, the system cannot progress beyond a short-lived demo.

There is also a security-related weakness in many existing student-level systems. Authentication may be superficial, password reset is often absent, session tracking is missing, and there is little thought given to account lockout, breach detection, or two-factor security. For a financial or finance-like application, this is a serious gap. FinHQ responds to this by explicitly building secure user-management logic into the backend.

1.3 OBJECTIVES

To design and implement a secure, modular, and data-centric platform that unifies investment analytics, market research, paper trading, alerting, and AI-assisted financial decision support within one extensible system.

1.4 PURPOSE, SCOPE AND APPLICABILITY

1.4.1 Purpose

The purpose of the project is to show how a financial market decision-support platform can be designed using structured software-engineering principles and modern web technologies. The project aims to improve on fragmented financial workflows by integrating market data, portfolio intelligence, research support, and analytics inside one environment.

The project is also academically significant because it demonstrates full-stack reasoning. It is not limited to one algorithm, one page, or one API. It includes architecture, data models, service decomposition, authentication, analytics, user flows, and deployment. As a result, it allows evaluation from multiple perspectives: design quality, implementation quality, security awareness, and future scalability.

1.4.2 Scope

The scope of the project includes:

- frontend implementation for authentication and analytics-oriented routes;
- backend APIs for auth, stocks, trading, alerts, social, strategies, and education;
- machine-learning service support for prediction, risk, recommendation, backtesting, fundamentals, and pattern detection;
- database-backed domain entities for user state, portfolios, orders, alerts, and content;
- Docker-based environment setup for local deployment;
- technical documentation sufficient for academic review and future enhancement planning.

The scope excludes live capital trading, complete production deployment, and full replacement of all staged or mock frontend values. Some modules are already wired to backend and ML services, while others still present representative UI data. This final report documents both implemented capability and current boundary conditions.

1.4.3 Applicability

The project is directly applicable to educational fintech environments, stock market training systems, and prototype investment-support applications. It can also serve as a basis for future internal analytics platforms, research dashboards, or student learning environments.

Indirectly, the system supports financial literacy and evidence-based decision-making. Users can move from raw market data to interpreted metrics, from portfolio values to risk views, and from news headlines to sentiment-enriched context. This makes the project useful not only as software but also as a structured decision-support environment.

1.5 OVERVIEW OF THE REPORT

This report is structured into six chapters. Chapter 1 introduces the project and explains its purpose, scope, and applicability. Chapter 2 analyzes the problem and defines system requirements, user categories, constraints, and conceptual models. Chapter 3 explains architecture, modules, database design, interface logic, and report-design considerations. Chapter 4 presents the implementation approach, coding standards, and selected implementation details. Chapter 5 discusses test cases, testing approaches, and expected reports. Chapter 6 concludes the report by discussing design and implementation issues, advantages, limitations, and future scope. Appendices and references follow the main chapters.

CHAPTER 2

SYSTEM ANALYSIS AND REQUIREMENTS

This chapter defines the system problem in analytical terms and identifies the requirements required for the solution.

2.1 PROBLEM DEFINITION

The project addresses the problem of fragmented and weakly integrated investment-support tooling. In many real or prototype environments, the user must manually combine charting, research, alerts, news, and account interpretation. This leads to duplicated effort, poor continuity, and delayed decision-making.

From a software perspective, the broader problem includes:

1. absence of unified access to market data, analysis, and action-oriented workflows;
2. limited security and identity management in educational or prototype finance applications;
3. poor separation between core business logic and analytics-heavy computation;
4. insufficient data modeling for portfolios, alerts, transactions, and user state;
5. lack of explainable, modular AI integration in financial applications.

The project therefore solves not only a user problem but also a design problem. It aims to create a system in which data flows, business logic, and analytical intelligence are placed within a clear architecture that can evolve over time.

2.2 REQUIREMENTS SPECIFICATION

The requirements of the system are derived from the implementation present in the repository and from the practical needs of a decision-support-oriented fintech platform [1].

2.2.1 Functional Requirements

- FR1. The system shall support user registration with password validation.
- FR2. The system shall support secure login and refresh-token flow.
- FR3. The system shall support password reset and password-history validation.
- FR4. The system shall support optional two-factor authentication workflows.
- FR5. The system shall allow users to search stocks by ticker or company name.
- FR6. The system shall provide historical OHLCV market data.
- FR7. The system shall compute and provide technical indicators.
- FR8. The system shall display quote, fundamentals, and market news.
- FR9. The system shall support creation and viewing of paper-trading orders.
- FR10. The system shall support positions and portfolio-summary retrieval.
- FR11. The system shall support alert creation, alert history, and notification state.
- FR12. The system shall support strategy storage and backtesting-oriented workflows.
- FR13. The system shall support educational module access and progress recording.
- FR14. The system shall support social posting and commenting.
- FR15. The system shall expose ML endpoints for prediction, recommendation, sentiment, risk, backtesting, and pattern detection.
- FR16. The system shall support WebSocket-based live or simulated update channels.

2.2.2 Non-Functional Requirements

- NFR1. The system shall maintain strong security for account-related workflows.
- NFR2. The architecture shall remain modular and service-oriented.
- NFR3. The codebase shall remain understandable and maintainable.
- NFR4. The system shall support local reproducibility through Docker-based deployment.
- NFR5. The system shall allow future integration of live data and broker connectivity.
- NFR6. The system shall maintain clear separation between transactional and analytical workloads.

NFR7. The system shall remain demonstrable even where some modules use staged UI data.

NFR8. The documentation shall distinguish between implemented and partially integrated functionality.

2.3 BLOCK DIAGRAM

The overall block diagram is shown in Fig. 2.1.

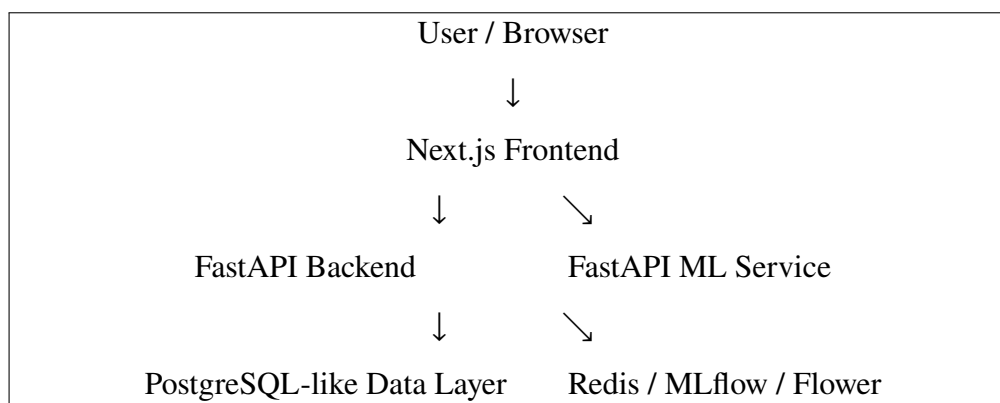


Figure 2.1: High-level block diagram of FinHQ

This block diagram makes visible the most important design principle of the system: a dedicated analytics service exists alongside, not inside, the main transactional backend. That separation improves clarity, maintainability, and future scalability.

2.4 SYSTEM REQUIREMENTS

2.4.1 User Characteristics

Table 2.1: User characteristics

User Category	Characteristics
Student / Learner	Needs guided market understanding, safe experimentation, and explanatory outputs.
Research-Oriented Investor	Needs quick access to charts, quotes, indicators, fundamentals, and news.
Developer / Maintainer	Needs modular source organization, service boundaries, and reproducible setup.
Academic Evaluator	Needs evidence of proper architecture, documentation, implementation depth, and future scope.

2.4.2 Software and Hardware Requirements

Table 2.2: Software requirements

Requirement Area	Details
Operating System	Windows, macOS, or Linux with modern development tooling.
Frontend Software	Node.js ecosystem for Next.js, React, TypeScript, and package management [2], [3].
Backend Software	Python 3.11, FastAPI, SQLAlchemy, and related dependencies [4].
Database Software	PostgreSQL-compatible relational database [6].
Containerization	Docker and Docker Compose [5].
Analytics Stack	PyTorch, scikit-learn, pandas, numpy, MLflow [7], [8].
Developer Tools	Git, editor or IDE, terminal, and browser-based validation tools.

Table 2.3: Indicative hardware requirements

Hardware Item	Indicative Requirement
Processor	Multi-core processor for concurrent service execution.
RAM	8 GB minimum; 16 GB preferred for smoother multi-container and ML tasks.
Storage	Sufficient storage for source code, containers, database, and artifacts.
Network	Internet access for dependency installation and optional data-provider access.
Display	Standard desktop or laptop display; wider displays improve analytics workflows.

2.4.3 Constraints

The following constraints influence the system:

- not all UI routes are fully integrated with live backend data at this stage;
- broker execution remains simulated and not real;

- some external data-provider behavior may vary;
- environment complexity increases due to multiple coordinated services;
- production-level observability, secrets handling, and governance are not yet complete;
- model effectiveness depends on dataset quality and further training cycles.

2.5 CONCEPTUAL MODELS

2.5.1 Data Flow Diagram

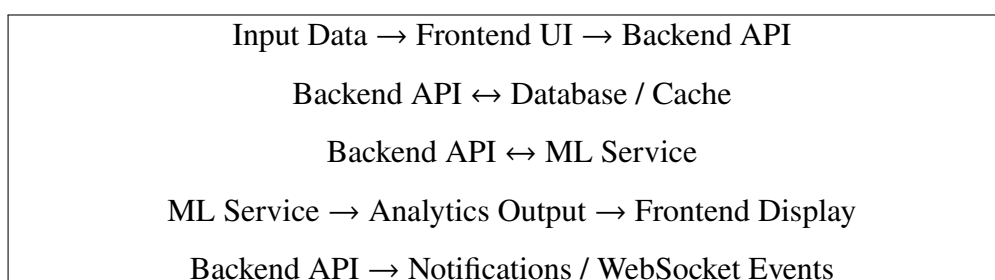


Figure 2.2: Conceptual data flow of the system

2.5.2 ER Diagram

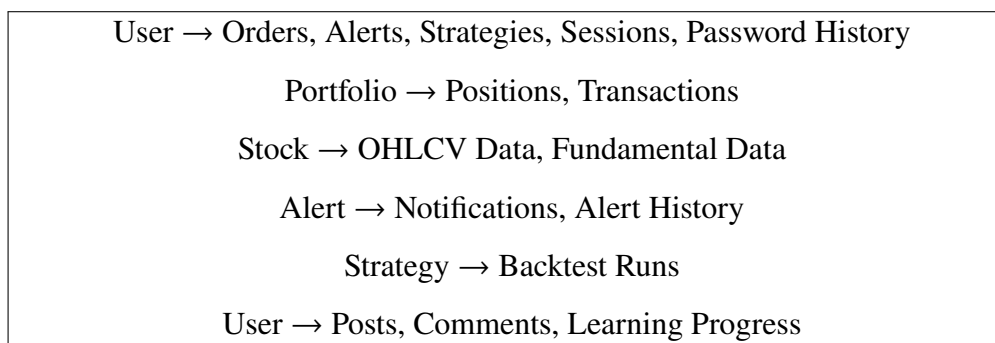


Figure 2.3: Conceptual entity relationship view

The conceptual models show that the application is not centered only on market data. It also models user identity, transactional state, analytical output, and user-driven engagement.

CHAPTER 3

SYSTEM DESIGN

This chapter discusses the structure of the system in terms of architecture, modules, database reasoning, interface design, and procedural flow.

3.1 SYSTEM ARCHITECTURE

FinHQ uses a layered, multi-service architecture. The presentation layer is implemented through a Next.js application that organizes user functionality into route groups such as authentication and dashboard-driven modules. The application logic layer is implemented through a FastAPI backend that contains domain-specific routes and service classes. The analytics layer is implemented through a separate FastAPI ML microservice. The data layer is composed of PostgreSQL-oriented persistence and Redis-backed support. Supporting infrastructure includes Docker Compose, MLflow, and Flower.

This architectural pattern is especially suitable for a final report because it illustrates not only what the project does, but how it is organized. The architecture makes each layer easier to evolve independently. User interface changes can occur without rewriting the ML service. New analytics can be added without directly changing authentication logic. Database models can expand as the feature set grows.

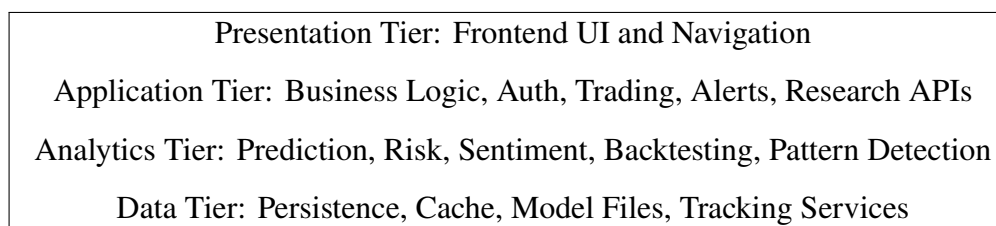


Figure 3.1: System architecture view

3.2 MODULE DESIGN

The overall system has been divided into modules following a divide-and-conquer strategy.

3.2.1 Authentication and Identity Module

This module provides registration, login, refresh-token support, password reset, password-history control, and TOTP-oriented 2FA support. Its design emphasizes security and state consistency. The module includes frontend forms and backend validation logic. It is tightly connected to user sessions, password history, and access control.

3.2.2 Dashboard and Portfolio Module

This module gives the user an executive view of account status. The dashboard brings together balance, allocation, alerts, movers, and briefing information. The portfolio module expands that into holdings and recent activity. These modules are essential because they transform raw data into actionable overview.

3.2.3 Technical Analysis Module

This module supports chart-based interpretation of securities. It includes stock search, quote retrieval, OHLCV display, indicator overlays, and AI-backed pattern support. It combines backend-calculated indicators with frontend visualization and thereby demonstrates cross-layer cooperation.

3.2.4 Fundamental Analysis and News Module

This module supports research-oriented asset evaluation. It includes company metrics, valuation assumptions, ratio display, market news, and sentiment-style interpretation. It moves the user from price tracking into broader analytical reasoning.

3.2.5 Trading, Alerts, and Strategy Module

This module supports paper trading, order state, user alerts, and strategy management. It is important because it represents the action side of the system. Research and analysis become useful only when the user can convert them into tracked decisions or monitored conditions.

3.2.6 Community and Learning Module

This module extends the system beyond narrow analytics. Community features allow user-driven content and interaction, while the learning module supports educational progress. These features improve applicability for students and evaluation environments.

3.2.7 ML and Quantitative Module

This module contains feature generation, forecasting, risk metrics, recommendation support, sentiment analysis, backtesting logic, and pattern detection. It is one of the defining characteristics of the project because it changes the platform from a simple market dashboard into a decision-support system.

3.3 DATABASE DESIGN

The database design is one of the central strengths of the repository because it maps application concepts to durable entities. This helps the system retain continuity between user actions, market context, and analytical output.

3.3.1 Tables and Relationships

Table 3.1: Key tables and relationships

Entity	Role in Database Design
User	Central identity object tied to security state and multiple domain relationships.
PasswordHistory	Protects password reuse and strengthens account integrity.
UserSession	Maintains session-level context and device information.
Stock / OHLCV / Fundamentals	Stores market identity and research-related data.
Portfolio / Position / Transaction	Represents cash, holdings, and trading ledger state.
Order / Trade	Represents order lifecycle and trading activity.
Strategy / BacktestRun	Stores strategy logic references and simulation result context.
Alert / Notification / AlertHistory	Models condition monitoring and generated events.
Post / Comment / Follow / Like	Supports user interaction and social feed design.
Learning Entities	Support educational content, lessons, quizzes, and progress.

3.3.2 Data Integrity and Constraints

The system uses validations and constraints at multiple layers:

- schema-level validation through typed request and response models;
- email uniqueness and authenticated ownership checks;
- password rules and password-history enforcement;
- enum-style state boundaries for orders, alerts, and risk-profile values;
- relationship-based grouping of portfolios, transactions, sessions, and notifications;
- explicit checks for buying power and share availability in the broker service.

These constraints are essential because the system is not stateless. It maintains evolving account-specific information. Data integrity therefore directly influences system reliability.

3.4 SYSTEM CONFIGURATION

The system configuration is container-oriented. Each primary concern is separated into its own service. The frontend container runs the user interface. The backend container runs the business API. The ML container runs the analytics API. The database container supports storage, while Redis supports fast state and infrastructure coordination. MLflow and Flower extend the environment toward experiment tracking and task monitoring.

3.5 INTERFACE AND PROCEDURAL DESIGN

3.5.1 User Interface Design

The user interface is designed around route-level clarity and workflow continuity. The application distinguishes authentication pages from the main dashboard experience. Once authenticated, the user is placed in a navigation structure that exposes major analytical and operational modules. This prevents the interface from becoming overloaded and supports a sequential way of thinking: first access, then understand, then decide, then act or monitor.

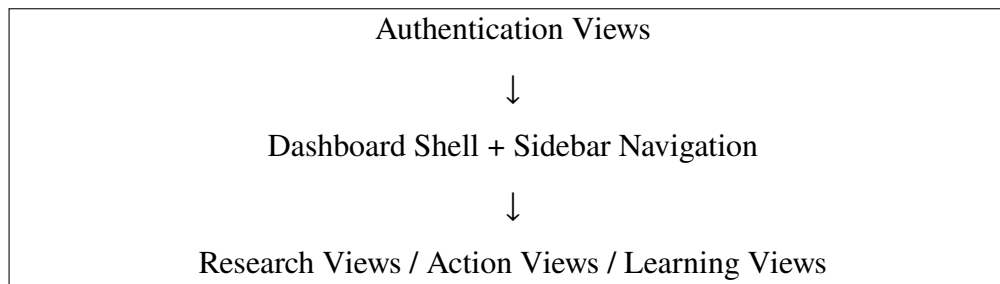


Figure 3.2: Conceptual UI navigation structure

3.5.2 Application Flow / Class Diagram

The procedure of the application can be summarized as:

1. the user authenticates through protected routes;
2. the frontend requests protected data using token context;
3. the backend validates identity and routes the request to the relevant service;
4. persistence or external/provider data is retrieved as required;
5. analytical enrichment is requested from the ML service when needed;
6. the frontend renders the final result as chart, card, table, or action form;
7. the user may then create an order, alert, or strategy based on the observed information.

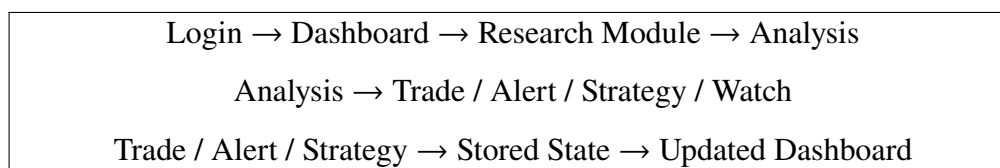


Figure 3.3: Application flow of the platform

3.6 REPORTS DESIGN

In the context of this project, report generation can be interpreted in two ways. The first is academic project documentation such as this report. The second is system-generated financial or analytical reports that could be offered to the user. The platform's data model and module structure are suitable for generating:

- portfolio summary reports;
- transaction reports;
- alert activity reports;
- strategy and backtesting reports;
- stock analysis reports;
- risk reports;
- user-progress reports in the learning module.

Inputs to such reports may include date range, asset symbol, strategy identifier, user identity, or selected portfolio. Output fields would vary by report type but can include timestamps, prices, indicator values, trade details, trigger events, or ML-generated metrics.

CHAPTER 4

IMPLEMENTATION

This chapter explains how the design has been converted into a working implementation.

4.1 IMPLEMENTATION APPROACHES

The implementation follows an incremental modular approach. The platform has not been treated as a single coding artifact; rather, it has been developed through interacting modules. Frontend pages were created to reflect user workflows. Backend routes and services were added to support protected and domain-specific logic. ML services were created separately for analytics-heavy functions.

This approach has several benefits. It allows the project to grow progressively without forcing a redesign of earlier work. It also makes debugging more manageable because problems can often be traced to one layer or one domain module. The cost of this approach is higher integration complexity, but the architectural gains justify that trade-off.

4.2 CODING STANDARD

The repository reflects the use of practical coding standards and structure:

- logical file grouping by concern in frontend, backend, and ML service;
- route-based organization in the frontend;
- clear separation of endpoints, services, models, and schemas in the backend;
- use of async patterns in backend services;
- consistent naming of service classes and route handlers;
- type usage in frontend and structured models in Python;
- framework-aligned coding conventions using official ecosystem practices [2]–[8].

4.3 CODING DETAILS

4.3.1 Authentication and Security Implementation

The authentication implementation is one of the most significant parts of the project. Registration validates password quality and checks whether the password has appeared in known breaches. Password hashes are stored using Argon2-compatible logic. Failed logins increase a counter and eventually trigger lockout behavior. Password resets also verify that a new password does not reuse recent credentials. Two-factor authentication is supported through TOTP-related logic at the backend level.

4.3.2 Market Data Implementation

The market data implementation combines database-backed retrieval with fallback or provider-oriented retrieval. The stock service can search symbols, fetch OHLCV history, compute server-side indicators, return live quote information, and provide fundamentals and news. This gives the project a practical research backbone.

4.3.3 Trading and Alert Implementation

The broker service creates or loads a portfolio, checks available balance, simulates fill behavior, updates position state, and stores transactions. Alert logic watches for threshold conditions and records notification and history events once a trigger occurs. This gives the system a meaningful notion of stateful user action and consequence.

4.3.4 ML Implementation

The ML implementation is service-oriented. The prediction service uses sequence modeling for next-price estimation. The risk service aligns return data and computes portfolio statistics. The pattern-detection service combines algorithmic pattern logic with deeper-learning-oriented structures. The sentiment service and recommendation logic extend the system into interpretive AI. This breadth is a major reason the report classifies the project as a decision-support platform rather than a basic data viewer.

4.4 SCREEN SHOTS

For the final report, screenshot placeholders are retained so the document can later be completed with final captured UI states if institution-specific screenshots are added at submission time.

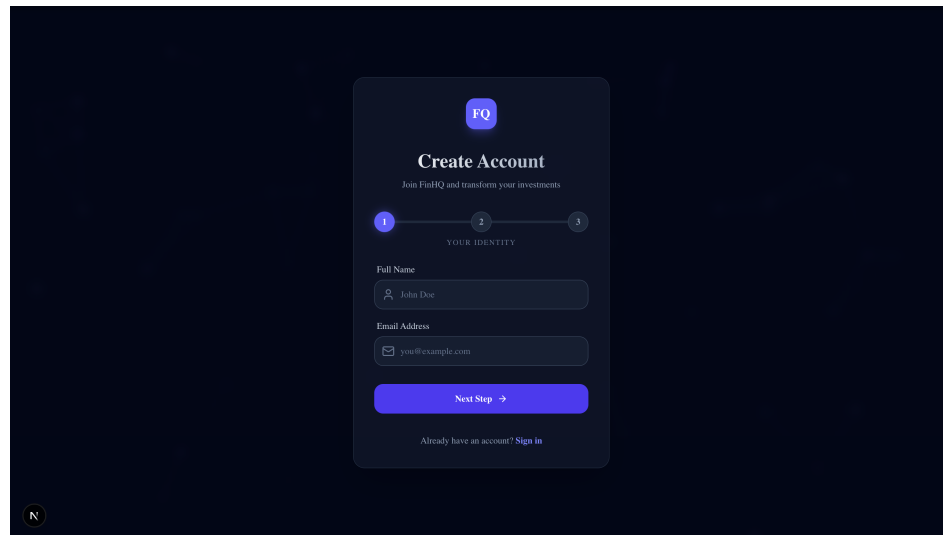


Figure 4.1: Registration Screen

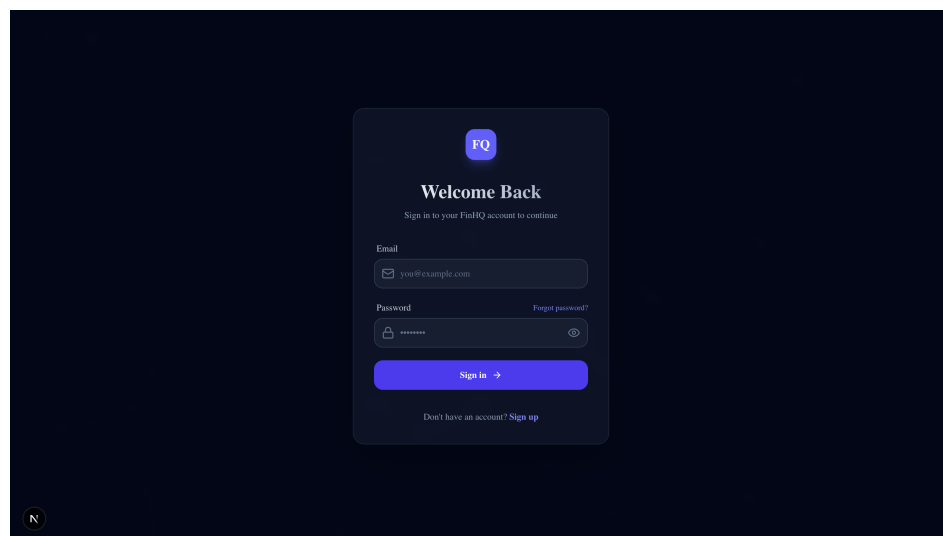


Figure 4.2: Login Screen

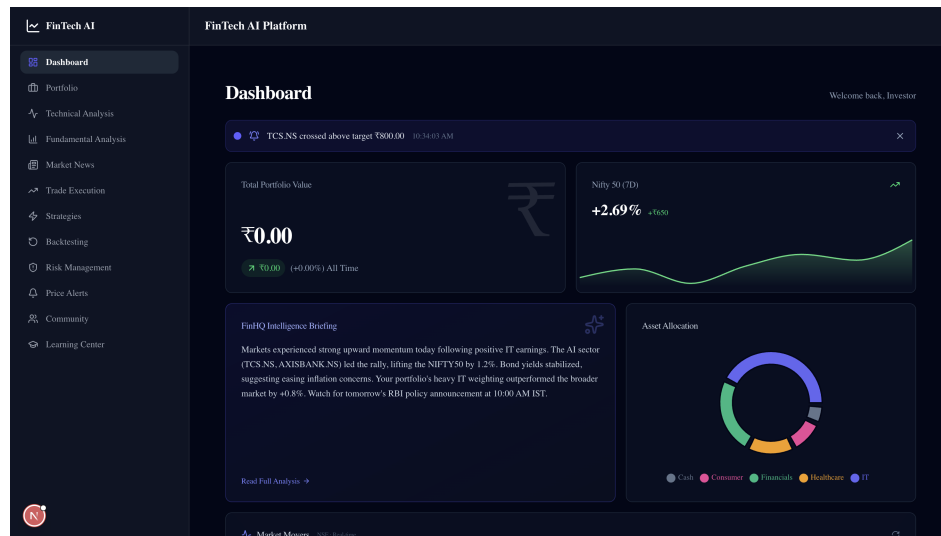


Figure 4.3: Main Dashboard

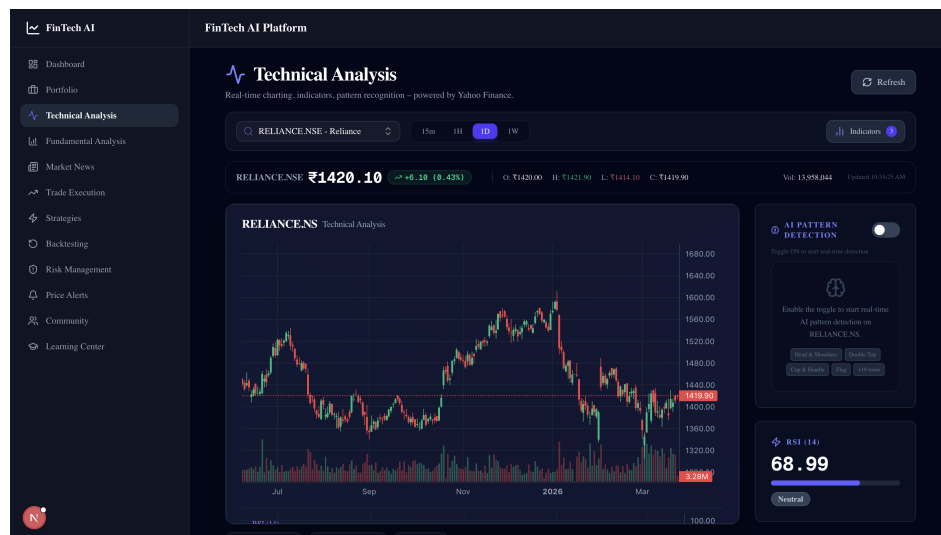


Figure 4.4: Technical Analysis

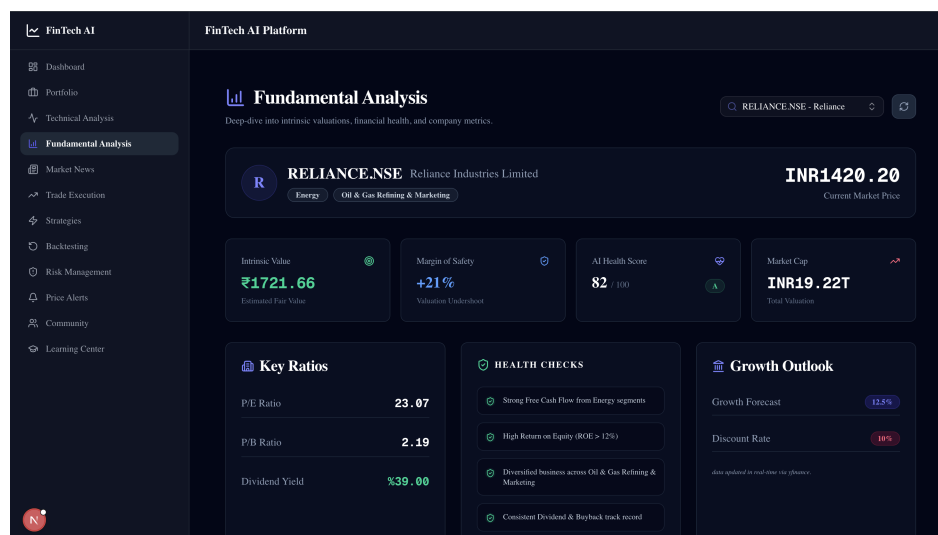


Figure 4.5: Fundamental Analysis

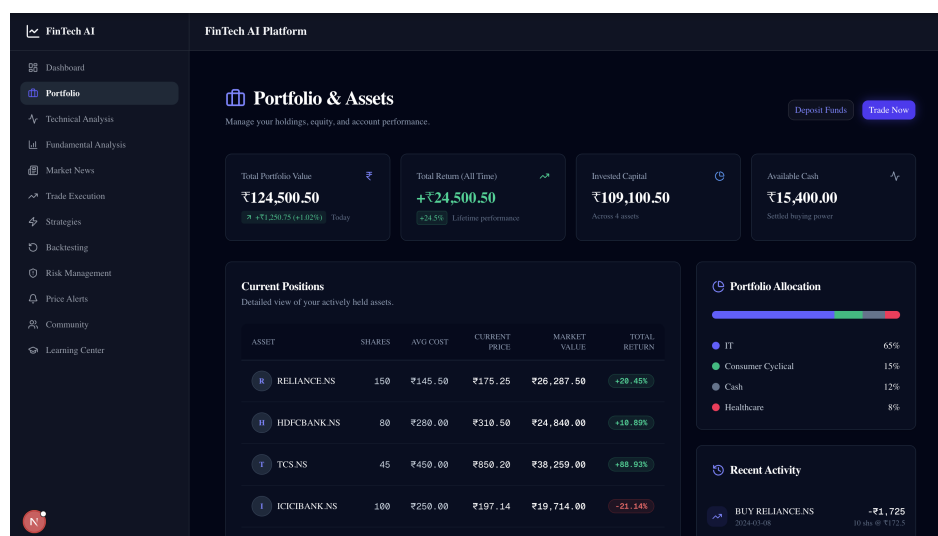


Figure 4.6: Portfolio Management

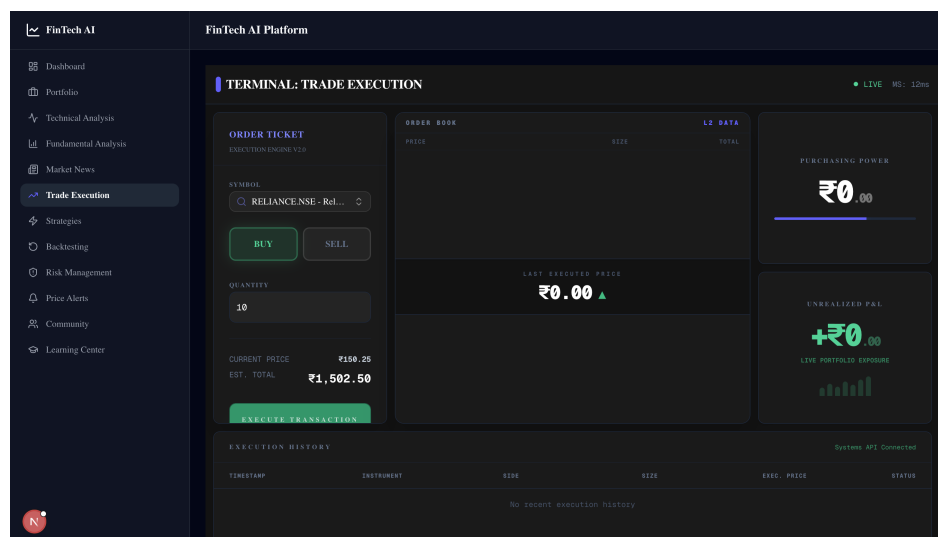


Figure 4.7: Trading Interface

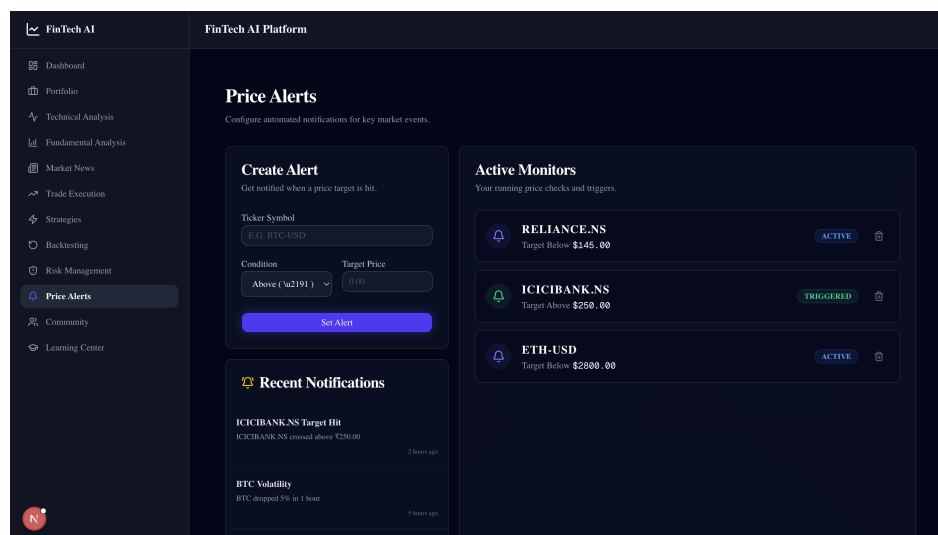


Figure 4.8: Alerts and Notifications

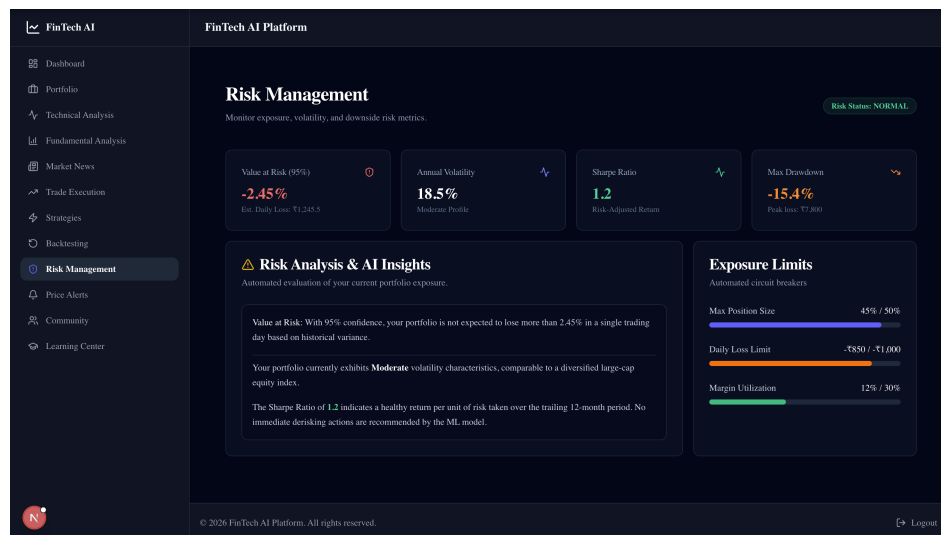


Figure 4.9: Risk Management

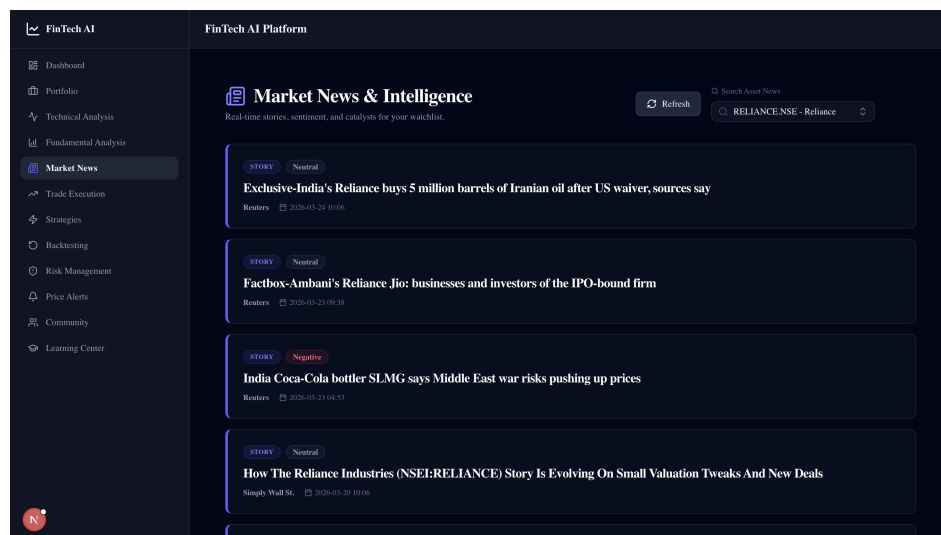


Figure 4.10: Market News

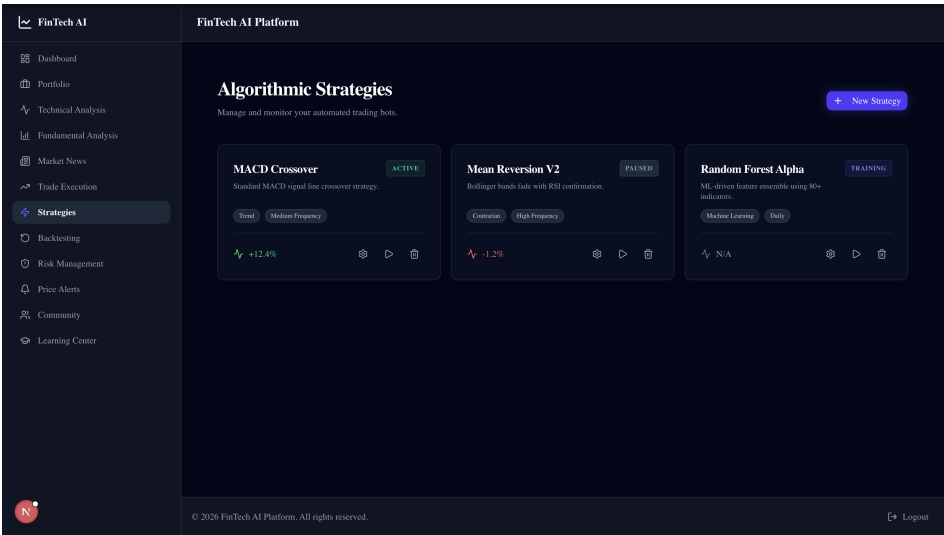


Figure 4.11: Strategy Builder

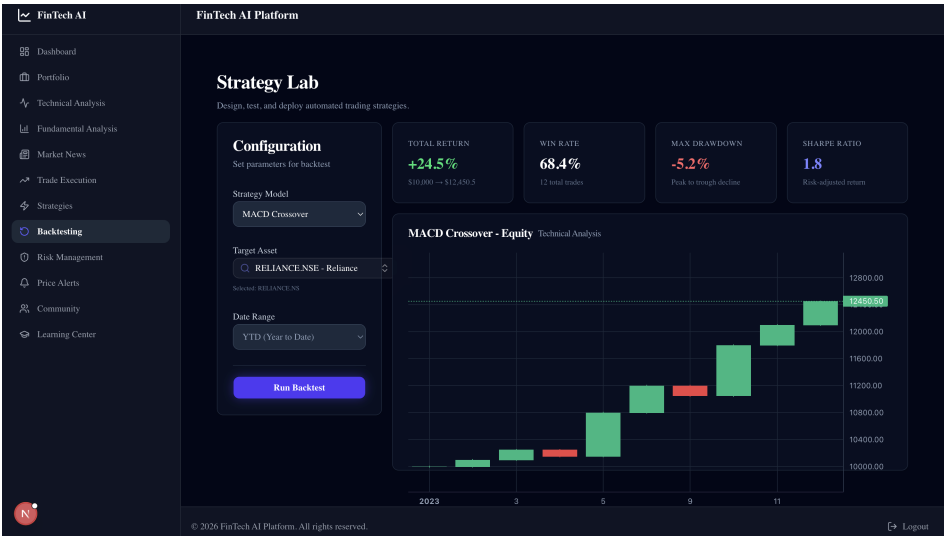


Figure 4.12: Strategy Backtesting

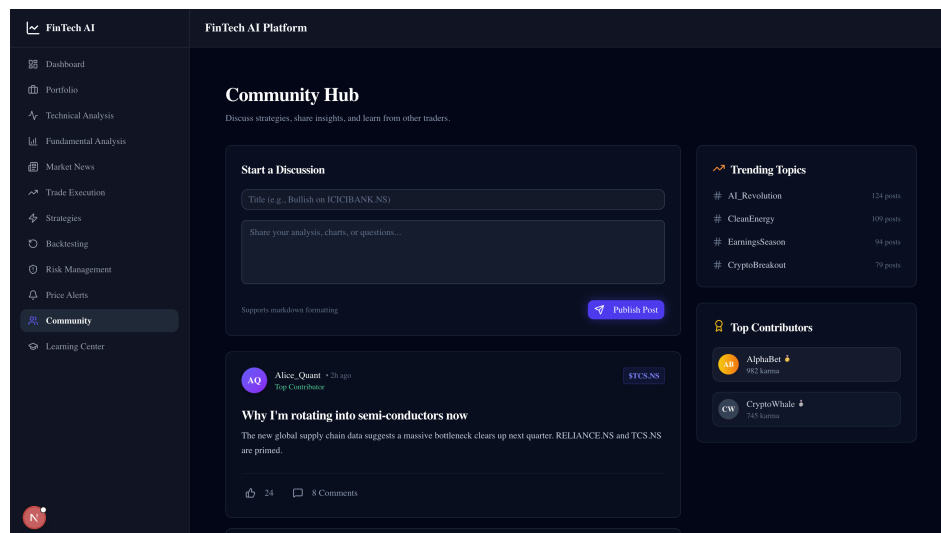


Figure 4.13: Community Support

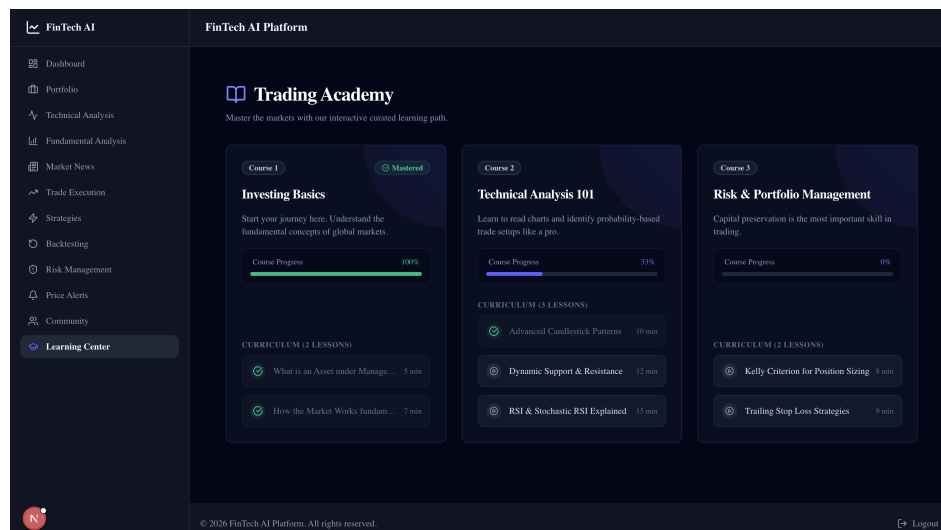


Figure 4.14: Learning Center

CHAPTER 5

TESTING

This chapter discusses how the requirements can be validated and what test evidence is needed for the project.

5.1 TEST CASES

Table 5.1: Expanded representative test cases

TC No.	Scenario	Expected Result
TC1	Register with valid credentials	User created and stored successfully.
TC2	Register with weak password	Registration rejected with clear validation response.
TC3	Register with breached password	Registration rejected due to breach-check result.
TC4	Successful login	Access granted and token response returned.
TC5	Invalid login	Error returned and failed-attempt count increased.
TC6	Excess invalid login attempts	Temporary account lockout applied.
TC7	Reset password using old password	Reset rejected based on password history.
TC8	Fetch stock search result	Matching symbols returned in bounded list.
TC9	Fetch OHLCV and indicators	Candle and indicator payloads returned in usable format.
TC10	Create valid buy order	Order stored and position state updated.
TC11	Sell without shares	Request rejected with share-availability error.
TC12	Create alert and cross threshold	Triggered notification and alert history created.
TC13	Fetch risk analysis	Portfolio risk metrics returned.
TC14	Run pattern detection on OHLCV data	Pattern list and optional annotation returned.
TC15	Access protected route without auth token	Unauthorized response generated.

5.2 TESTING APPROACHES

5.2.1 Unit Testing

Unit testing should target service-level methods such as password validation, token creation, order fill simulation, indicator generation, and risk calculations. Because the code is already organized into service classes, the project is structurally suitable for such testing.

5.2.2 Integration Testing

Integration testing should validate route-to-service-to-database interaction. Strong candidates include auth flows, reset flows, stock and indicator retrieval, order placement and position retrieval, and alert creation followed by trigger validation.

5.2.3 System and Workflow Testing

System testing must verify that route-level navigation and cross-module workflows behave as intended. For example, a user should be able to log in, reach a protected dashboard, inspect technical analysis, and create an alert or strategy without encountering broken flow. This is especially important because some UI modules are already integrated while others are partially staged.

5.3 TEST REPORTS

Table 5.2: Revised test report view

Test Domain	Revised Observation
Authentication	Security logic is robust enough to justify deeper integration testing.
Data Retrieval	Stock search, quote, OHLCV, and indicator routes provide a concrete basis for API validation.
Trading	State transitions are well defined but need deterministic testing scenarios.
Alerts	Trigger-based behavior is clear and suitable for event-flow testing.
ML APIs	Contracts exist for multiple analytics workflows and should be validated with fixture data.
Frontend Workflows	A mixed pattern exists: some pages are wired to APIs while others use staged data and need post-integration retesting.

The final testing conclusion is that the architecture supports systematic testing, but the project would benefit from a larger explicitly maintained automated suite before any production-level claim is made.

CHAPTER 6

CONCLUSION

This chapter presents the main issues, strengths, limitations, and future directions of the project.

6.1 DESIGN AND IMPLEMENTATION ISSUES

One of the main design issues in the project is controlled scope management. Since the platform covers authentication, research, trading, alerts, learning, community interaction, and AI services, there is a constant risk of feature spread. The architectural separation into modules helps manage that risk, but it also introduces a coordination challenge between layers.

Another issue is balancing usability and depth. Financial applications can become too complex for users if every metric is exposed at once. The route-based UI and card-oriented presentation help control this, but further refinement is still possible, especially once all modules are connected to live state.

Implementation issues include multi-container setup complexity, dependency coordination, and the need to maintain consistency between backend logic and frontend behavior. Where staged UI data exists, the challenge is not designing the screen but integrating it without breaking design coherence.

6.2 ADVANTAGES AND LIMITATIONS

6.2.1 Advantages

- The project provides strong breadth across multiple fintech workflows.
- The architecture is modular and demonstrates mature software decomposition.
- Authentication and account-hardening features are stronger than many student-level finance prototypes.
- The ML component is integrated into the platform as an analytical service layer.
- The database and domain model support future extension and richer reporting.
- The project is useful for demonstration, teaching, and further software engineering work.

6.2.2 Limitations

- Some screens still use mock or representative data.
- Live brokerage connectivity is absent.
- Automated tests are not yet extensive enough for stronger quality claims.
- Some frontend flows need tighter alignment with secure backend logic.
- Production deployment, secrets handling, and observability still need improvement.
- Model performance and explainability require further refinement and dataset work.

6.3 FUTURE SCOPE OF THE PROJECT

The future scope of FinHQ includes three major directions. The first is deeper integration: all visible UI modules should use validated live backend or ML responses wherever possible. The second is stronger quality assurance through automated tests, CI, and deployment verification. The third is capability expansion through live broker APIs, richer research datasets, more advanced risk simulation, improved model governance, and broader user personalization.

In summary, the aim of the project was to build a data-centric platform for investment analytics and financial market decision support. The implemented system achieves this in a meaningful way through secure user management, data-rich research workflows, modular analytics services, and clear system architecture. The major modules are present and coherent, the limitations are known, and the future direction is realistic.

CHAPTER 7

DETAILED MODULE AND FUNCTIONAL CATALOG

This chapter expands the module-level view of the project and documents the system in a deeper functional manner than the earlier chapters.

7.1 FRONTEND MODULE CATALOG

The frontend of FinHQ is implemented using a route-oriented structure in Next.js and React [2], [3]. This structure is important because each major business capability is represented as an explicit user journey rather than being hidden inside a generic single-page interface. The main functional groups are authentication routes and dashboard-oriented protected routes.

7.1.1 Authentication Routes

The authentication experience includes login, registration, forgot-password, reset-password, and 2FA setup flows. The login route accepts user credentials and supports conditional two-factor verification. The registration route uses a multi-step form so that user identity, password quality, and investment preference can be captured without making the process feel abrupt. Forgot-password and reset-password routes translate backend security policies into accessible UI forms. The 2FA setup route introduces the user to second-factor protection and represents a strong security-oriented capability for a student project.

7.1.2 Dashboard and Portfolio Routes

The dashboard route serves as the executive landing screen after authentication. It brings together account overview, allocation, movers, alerts, and AI-style summary widgets. The portfolio route expands this into a holdings-focused view with allocation logic and recent activity. These screens are important because they reflect the system's ability to move from raw values into decision-oriented summaries.

7.1.3 Analysis Routes

The analysis-oriented routes include technical analysis, fundamental analysis, news, and risk. The technical route offers chart-centric exploration and indicator overlays. The fundamental route supports valuation, score-based interpretation, and company context. The news route surfaces recent asset-related stories and sentiment cues. The risk route expresses exposure and portfolio interpretation through a metrics-oriented layout. Together, these routes form the research core of the platform.

7.1.4 Action and Simulation Routes

The action-oriented routes include trade, strategies, backtest, and alerts. The trading route allows the user to convert a market view into a simulated order. The strategies route stores and presents strategy definitions. The backtest route allows strategy evaluation against historical data. The alerts route lets the user convert passive observation into monitored market conditions. These routes make the project more than a reporting dashboard.

7.1.5 Engagement and Learning Routes

The community route introduces collaboration and user-generated content, while the learning route supports modular educational progression. These routes are valuable because they broaden the system's educational and social applicability, which is especially useful in university or training contexts.

7.2 BACKEND FUNCTIONAL CATALOG

The backend is structured around domain routers and service classes, making the platform easier to reason about and extend.

7.2.1 Authentication and User Management

The authentication router covers registration, login, token refresh, two-factor setup, and two-factor verification. The user router includes profile access and password-reset-related flows. Security logic includes password complexity enforcement, breach detection, account lockout, password history checks, and session invalidation. This area is one of the strongest indicators of engineering maturity in the repository.

7.2.2 Market and Research Services

The stock router and stock-data service together provide search, quote, OHLCV, indicator, fundamental, and news functionality. This means the backend is not merely a CRUD API over database records; it is an orchestration layer for market-facing workflows.

7.2.3 Trading and Alert Services

The trading router works with the broker service to create orders, validate buying power, update positions, and return portfolio views. The alert router works with alert services and history tracking to support threshold-based monitoring. Together these services handle stateful user actions.

7.2.4 Strategies, Education, and Social Services

Strategies are managed through dedicated CRUD endpoints. Educational flows provide seeded modules and lesson-completion support. Social flows provide post and comment functionality. These services reveal that the platform has been designed with broader ecosystem thinking rather than a narrow charting focus.

7.3 ML FUNCTIONAL CATALOG

The ML layer contributes to the analytical identity of the system.

7.3.1 Feature Engineering

Feature engineering generates technical indicators, candlestick-like features, and support or resistance outputs from market series. This helps bridge raw OHLCV data and model-ready numerical representations.

7.3.2 Forecasting

The prediction service trains and serves an LSTM-based sequence model. The choice of a sequence model is appropriate because market prices are time-dependent and benefit from contextual historical windows. Although the project remains a draft-level platform rather than a production forecasting engine, the inclusion of a train-and-forecast path is valuable.

7.3.3 Risk Analytics

The risk service calculates portfolio-level statistics such as volatility, VaR, Sharpe ratio, and maximum drawdown. These are among the most relevant metrics for educational risk interpretation and portfolio analysis.

7.3.4 Recommendation and Sentiment

Recommendation logic provides decision-support output derived from data and optional textual context. Sentiment logic adds text-based market interpretation. These services enrich the application beyond numerical charting.

7.3.5 Backtesting and Pattern Detection

Backtesting evaluates strategy behavior over historical periods. Pattern detection combines algorithmic logic with deeper model-oriented approaches. This is especially relevant in technical-analysis-oriented fintech applications, where recognition of chart structures can guide user attention.

CHAPTER 8

DATA-CENTRIC ARCHITECTURE AND DATA MANAGEMENT

This chapter focuses on the project's data-centric character and explains why data design is central to the platform.

8.1 WHY THE PLATFORM IS DATA-CENTRIC

The title of the project emphasizes that the platform is data-centric, and this description is justified by the architecture. Almost every major function of the system depends on acquiring, transforming, storing, or interpreting structured data. User identity data supports authentication and personalization. Market data supports charting and analysis. Transactional data supports paper trading and portfolio reporting. Event data supports alerting and notifications. Educational data supports content progress. Analytical data supports prediction, recommendation, risk, and sentiment interpretation.

The system is therefore not simply a visually rich application. Its design revolves around a chain of data operations: ingestion, normalization, validation, persistence, aggregation, enrichment, and presentation. This is exactly the type of structural thinking that distinguishes a data-centric platform from a page-centric application.

8.2 PRIMARY DATA DOMAINS

The platform contains at least five major data domains:

1. identity and account-security data;
2. asset and market-series data;
3. portfolio and transaction-state data;
4. event and notification data;
5. analytics and model-output data.

These domains are not isolated; they interact continuously. For example, a logged-in user's portfolio influences dashboard displays, alert thresholds, and risk calculations. Market-series data informs indicator logic, model features, and chart-based pattern

detection. This interconnectedness is why robust data modeling is so important in the project.

8.3 DATA FLOW ACROSS LAYERS

At the presentation layer, data is requested in a user-centric form. At the backend layer, that same data is validated and aligned with business rules. At the analytics layer, market data may be transformed into features or predictions. At the persistence layer, the system stores state and history so that the platform remains coherent across sessions. This layered data flow reduces ambiguity and improves maintainability.

8.4 DATA QUALITY CONSIDERATIONS

Data quality is a major concern in any financial system. Missing values, stale data, inconsistent provider responses, and user-state mismatches can all reduce trust. The architecture of FinHQ helps manage these risks by using service abstraction, typed schemas, and modular validation. However, the final production quality of data still depends on stronger test coverage, operational monitoring, and live-source governance.

8.5 DATA SECURITY AND PRIVACY

Because the system contains account data, session information, portfolio state, and user-generated content, data protection is essential. The implemented security controls reduce exposure at the application level, but future versions should also strengthen operational controls such as secrets management, environment segregation, backup strategy, and audit logging.

CHAPTER 9

API CONTRACTS, ROUTES, AND INTEGRATION SURFACES

This chapter documents the system's interface surfaces in more narrative detail.

9.1 BACKEND ROUTE FAMILIES

The backend API is grouped by domain. This is a deliberate and effective design decision.

9.1.1 Authentication Family

The authentication family covers registration, access token creation, 2FA branching, token refresh, and 2FA setup or verification. These routes are security-critical and define the entry boundary to the protected system.

9.1.2 User Family

The user family includes profile access and password-reset support. In a mature system, this family would likely grow to include preferences, profile edits, and session management views.

9.1.3 Stocks Family

The stock family is one of the richest route groups. It supports search, catalog access, stock details, OHLCV, indicators, quote retrieval, fundamentals, and news. In practical terms, this route family powers the research identity of the platform.

9.1.4 Trading Family

Trading routes provide order creation and retrieval, cancellation, position views, and portfolio summaries. These routes convert analytics into simulated user action.

9.1.5 Alerts, Strategy, Social, and Education Families

These route groups extend the platform from pure analytics into event monitoring, simulation logic, social engagement, and learning. Their inclusion is a sign that the platform aims to support continuous user interaction rather than one-time data lookup.

9.2 ML ROUTE FAMILIES

The ML routes are organized around analytical responsibilities. Unlike the backend routes, which are strongly tied to user state and domain entities, ML routes are tied more closely to computational tasks and analytical outputs. This distinction is healthy because it prevents transactional APIs from becoming overloaded with model logic.

9.3 API DESIGN STRENGTHS

The major strengths of the API design are:

- clear versioning;
- domain-based route grouping;
- protected dependencies for sensitive routes;
- separation between transactional and analytical contracts;
- support for future testing through predictable route organization.

9.4 INTEGRATION CHALLENGES

The main challenge is alignment. Since the UI, backend, and ML service evolve separately, each route family must be validated against frontend assumptions and data-shape expectations. This is especially important where some screens still use representative mock data.

CHAPTER 10

AI, QUANTITATIVE METHODS, AND MODELING STRATEGY

This chapter gives deeper attention to the project’s analytical and AI-oriented aspects.

10.1 ROLE OF AI IN THE PLATFORM

AI in FinHQ is not positioned as a single marketing label. Instead, it is distributed across several analytical services that each solve a different class of problem: prediction, risk, recommendation, sentiment, and pattern recognition. This service-based use of AI is preferable to one generic “AI” endpoint because it preserves clarity and separates different analytical concerns.

10.2 TIME-SERIES PREDICTION

The forecasting service uses an LSTM-based approach. Sequence models are appropriate when the temporal order of observations matters, which is the case for market data. The current implementation uses normalized close-price sequences and a fixed lookback window. While this is not a complete institutional-grade forecasting system, it is academically appropriate and technically credible for a multi-module platform.

10.3 RISK ANALYSIS

The risk component uses aligned asset returns to derive portfolio-level metrics. Metrics such as volatility, VaR, Sharpe ratio, and maximum drawdown are useful because they help translate a set of positions into interpretable portfolio risk. This is a necessary bridge between portfolio state and risk-aware decision support.

10.4 PATTERN DETECTION

The pattern-detection subsystem is one of the most interesting parts of the project. It combines algorithmic logic using peaks, troughs, and shape heuristics with model-oriented detection. This dual approach is valuable because it supports both rule-based interpretability and future statistical or learned improvements.

10.5 FUNDAMENTAL AND SENTIMENT ANALYSIS

The project does not limit analytics to price prediction. Fundamental analysis and sentiment-oriented logic broaden the scope. Fundamental services compute valuation and score-style outputs, while sentiment services interpret text inputs. This helps the platform bridge numerical analysis and qualitative market context.

10.6 MODEL GOVERNANCE AND FUTURE NEEDS

Even though the current project demonstrates strong analytical breadth, the final report must acknowledge that model governance is a future requirement. Version tracking, reproducible training, dataset documentation, validation methodology, and drift monitoring are important for any future production version. The presence of MLflow support is promising because it indicates awareness of lifecycle considerations [8].

CHAPTER 11

DEPLOYMENT, OPERATIONS, AND ENGINEERING MANAGEMENT

This chapter focuses on how the system is run, maintained, and evolved.

11.1 CONTAINERIZED DEPLOYMENT

Containerization is central to the project’s reproducibility. Each major concern is placed in its own runtime boundary. This allows environment setup to be documented clearly and makes local demonstration easier. In academic projects, this is especially valuable because it reduces “works only on one machine” risk.

11.2 SERVICE RESPONSIBILITIES

The frontend provides interaction, the backend governs protected business logic, the ML service governs analytics, the database stores durable state, Redis supports fast access, MLflow tracks model-related activity, and Flower supports monitoring. This separation makes the project easier to explain and evaluate.

11.3 OPERATIONS AND MAINTENANCE

Operationally, the platform requires health checks, log observation, environment-variable management, and dependency consistency. The more services a system has, the more important service health becomes. This is one reason why the final report must include operational guidance and not only architecture diagrams.

11.4 PROJECT MANAGEMENT VIEW

From a project-management standpoint, the repository reflects phased development: planning, auth and backend scaffolding, analytics integration, UI route expansion, and documentation refinement. A structured project like this benefits from milestone planning, interface freezing, and progressive integration validation.

11.5 QUALITY RISKS

Key operational risks include stale external data, service-start order issues, model dependency problems, and frontend-backend contract drift. These risks are manageable, but they justify stronger CI, automated testing, and monitoring as future improvements.

CHAPTER 12

COMPREHENSIVE EVALUATION

This chapter synthesizes the project from academic, technical, and product perspectives.

12.1 ACADEMIC VALUE

The project has strong academic value because it demonstrates architecture, database design, UI design, security, service decomposition, AI integration, and report preparation in one artifact. It is suitable for project viva and technical review because the codebase supports discussion at multiple levels of depth.

12.2 TECHNICAL VALUE

The strongest technical aspects are:

- secure backend auth design;
- modular route and service structure;
- broad domain model;
- ML-service separation;
- containerized reproducibility;
- range of financial workflows.

12.3 PRODUCT VALUE

From a product perspective, FinHQ already behaves like a coherent prototype platform rather than a disconnected collection of widgets. The user can authenticate, research, monitor, simulate, and interpret within one system. That is valuable even before live-market maturity is achieved.

12.4 LIMITATION SUMMARY

The most significant limitations are incomplete live integration across all UI screens, absence of real broker execution, limited visible automated testing, and the need for further ML validation and operational hardening. These limitations do not invalidate the project, but they define its current maturity boundary.

12.5 FINAL EVALUATION

The final evaluation is that FinHQ is a well-structured, data-centric fintech platform with meaningful implementation depth. It is professionally organized, technically credible, and capable of being extended into a more mature system. For academic purposes, it provides enough design, implementation, and analytical substance to justify a full final report.

CHAPTER A

USER'S MANUAL

A.1 INSTALLATION AND SETUP

The recommended setup begins by cloning the repository from <https://github.com/Abhijith-E/fintechphase2>. Docker and Docker Compose should be installed and active. After moving into the repository root, the user can start the services using the configured compose file. Once the frontend, backend, ML service, database, and infrastructure services are active, the user can access the web interface through the configured local frontend port.

A.2 BASIC USAGE FLOW

The first user journey is authentication. A new user registers, then logs in, and is redirected to the dashboard. From the dashboard the user can move to technical analysis, portfolio, fundamentals, news, alerts, and strategies. Trading and alert creation are available through dedicated screens, while educational and community content offer additional support.

A.3 MAINTENANCE GUIDANCE

Maintenance involves checking service health, container logs, database availability, cache availability, and environment variable correctness. When ML-related issues occur, the operator should verify model dependencies and service startup state. Since the project is multi-service, monitoring each component separately is important.

CHAPTER B

ADDITIONAL SUPPORTING TABLES

B.1 FRONTEND ROUTE INVENTORY

Route	Purpose
login	Secure login form with 2FA branching support.
register	Registration flow with password and profile capture.
forgot-password	Reset initiation route.
reset-password	Reset completion route.
2fa-setup	Two-factor setup user interface.
dashboard	Main post-login summary page.
portfolio	Portfolio and holdings visibility.
technical	Technical charting and indicator analysis.
fundamental	Fundamental and valuation review.
news	Asset-level news and sentiment-style display.
trade	Paper-trading screen.
strategies	Strategy listing and navigation.
backtest	Backtesting workspace.
risk	Risk and exposure analytics.
alerts	Alert creation and management.
community	Social interaction route.
learn	Learning-center route.

B.2 BACKEND SERVICE INVENTORY

Service	Purpose
StockDataService	Fetches and enriches stock-related data.
BrokerService	Handles paper-trading order and portfolio logic.
AlertService	Manages alerts, notifications, and trigger history.
EducationService	Manages education module access and progress.
SocialService	Supports social posts and comments.
WebSocket Connection-Manager	Handles subscriptions and event broadcast.

B.3 ML SERVICE INVENTORY

ML Service	Purpose
FeatureService	Technical feature and support/resistance computation.
PredictionService	Sequence-based price forecasting.
RecommendationService	Analytical recommendation generation.
RiskService	Portfolio risk metric computation.
FundamentalService	DCF and health-style scoring.
BacktestService	Strategy simulation support.
SentimentService	Financial text sentiment prediction.
TrainingService	Model-training-related workflows.
PatternDetectionService	Algorithmic and deep-learning pattern detection.
ChartRenderer	Pattern-related chart annotation.
PatternHistoryStore	Pattern history retention.

CHAPTER C

PLANNING AND DESIGN CONCEPTS

C.1 INDICATIVE GANTT-LIKE PLAN

Table C.1: Indicative phase-wise project progression

Phase	Activities
Planning	Problem understanding, scope definition, architecture planning, and module identification.
Implementation Phase I	Auth, core backend structure, frontend scaffolding, and database model setup.
Implementation Phase II	Market data, portfolio, trading, alerts, and strategy support.
Analytics Integration	Risk, prediction, fundamentals, pattern detection, and related ML services.
Testing and Documentation	Validation, issue tracking, report writing, and refinement.

C.2 DESIGN CONCEPTS CONSIDERATION

The project is software-centric, so health and safety concerns mainly translate into digital trust, security, and reliability. Security is addressed through account-hardening features. Usability is addressed through route-based workflow separation. Scalability is addressed through multi-service design. Maintainability is improved through domain-specific file organization and service decomposition.

CHAPTER D

EXTENDED FRONTEND AND SERVICE ANALYSIS

D.1 FRONTEND SCREEN ANALYSIS: LOGIN

The route ‘/login’ exists as part of the FinHQ user journey and should be understood as a task-focused entry point rather than a decorative screen. In implementation terms, it supports secure login, password entry, token-oriented authentication, and two-factor branching. Its architectural value lies in the fact that it translates backend security policy into an accessible user experience. The quality of this route directly affects the trustworthiness of the whole platform because every protected capability depends on it.

D.2 FRONTEND SCREEN ANALYSIS: REGISTER

The route ‘/register’ uses a structured multi-step approach to capture identity details, password quality, and initial profile settings. This is useful because registration in financial applications is more than account creation; it establishes the foundation for future user-specific decisions, alerts, sessions, and analytics.

D.3 FRONTEND SCREEN ANALYSIS: FORGOT PASSWORD AND RESET PASSWORD

The forgot-password and reset-password screens represent recovery-oriented security workflows. Their presence is important because many student projects ignore recovery scenarios. In FinHQ, recovery is documented as a first-class requirement. This improves completeness and shows awareness of real-world account lifecycle expectations.

D.4 FRONTEND SCREEN ANALYSIS: DASHBOARD

The dashboard route is the executive control panel of the platform. It consolidates balances, allocation, alerts, movers, and summary intelligence. Its existence demonstrates that the system has a top-level user view and is not merely a collection of disconnected pages. In product terms, this route translates multiple data sources into one decision-support surface.

D.5 FRONTEND SCREEN ANALYSIS: PORTFOLIO

The portfolio route is important because it grounds analytics in ownership and account state. Technical indicators and news become meaningful only when connected to holdings, cash, and performance. This screen therefore plays a crucial role in turning research into user-specific insight.

D.6 FRONTEND SCREEN ANALYSIS: TECHNICAL AND FUNDAMENTAL RESEARCH

The ‘technical’ and ‘fundamental’ routes represent two different but complementary forms of research. Technical analysis emphasizes price action, indicators, and chart structures, while fundamental analysis emphasizes company value, ratios, and research summaries. Their coexistence makes the platform more complete and educationally useful.

D.7 FRONTEND SCREEN ANALYSIS: NEWS, RISK, BACKTEST, TRADE, ALERTS, COMMUNITY, AND LEARN

The remaining research and action routes make the platform broad and cohesive. News connects market stories to asset context. Risk translates portfolio data into exposure metrics. Backtest allows strategy evaluation. Trade allows simulated execution. Alerts support monitoring. Community supports interaction. Learn supports knowledge building. This breadth is one of the defining strengths of the project.

D.8 BACKEND SERVICE ANALYSIS: STOCKDATASERVICE

StockDataService is one of the core service-layer abstractions in the backend. It fetches metadata, historical series, quotes, market movers, fundamentals, and enriched news. This is important because it places market-related logic in one coherent service layer instead of scattering it across many endpoints.

D.9 BACKEND SERVICE ANALYSIS: BROKERSERVICE

BrokerService manages portfolios, validates paper orders, simulates fills, updates positions, tracks cash, and handles cancellation. This service gives the platform an operational identity and demonstrates how user action can be modeled safely without live market execution.

D.10 BACKEND SERVICE ANALYSIS: ALERTSERVICE, EDUCATIONSERVICE, AND SOCIALSERVICE

AlertService adds event-driven monitoring. EducationService supports learning progression. SocialService supports posts and comments. These services show that the platform has broader user-life-cycle thinking and is not limited to chart display.

D.11 BACKEND SERVICE ANALYSIS: WEBSOCKET CONNECTION MANAGER

The WebSocket layer is strategically important because it prepares the system for event-driven interaction. Price streams, portfolio updates, and alerts can all be modeled as channels. Even where data is simulated, the presence of this live-communication layer strengthens the realism of the architecture.

D.12 ML SERVICE ANALYSIS

The ML services include feature engineering, forecasting, recommendation, risk, fundamental scoring, sentiment, training, pattern detection, chart rendering, and history storage. Their separation into distinct service units is one of the clearest signals that the AI portion of the platform has been thought through structurally rather than cosmetically.

CHAPTER E

EXTENDED API CONTRACT AND ENDPOINT ANALYSIS

E.1 AUTHENTICATION AND USER ENDPOINTS

The authentication endpoint family is critical because it defines who may enter the platform and how account state is protected. Register, login, 2FA verification, refresh, and password-reset flows together create the complete lifecycle of user access. This is stronger than a basic “username and password” demo because it includes multiple security checkpoints and state transitions.

E.2 STOCK AND RESEARCH ENDPOINTS

The stock endpoints provide search, market movers, catalog retrieval, stock details, OHLCV, indicators, quote data, fundamentals, and news. These routes form the data-delivery backbone of the research experience. In a final report, documenting them matters because they represent the interface between raw market data and user-facing analytics.

E.3 TRADING AND ALERT ENDPOINTS

Trading endpoints connect frontend user actions to portfolio state and order history. Alert endpoints connect market conditions to user notifications. Together, they make the system stateful and interactive. They are also useful from a testing perspective because they naturally define scenario-based flows.

E.4 STRATEGY, SOCIAL, AND EDUCATION ENDPOINTS

Strategy, social, and education endpoints demonstrate that the project is not limited to core financial CRUD operations. They allow the system to behave like an ecosystem platform with learning and engagement support.

E.5 ML ENDPOINTS

The ML endpoint surface is one of the most important differentiators of the project. Feature calculation, training, sentiment, prediction, recommendation, technical-analysis logic, risk estimation, fundamental analysis, backtesting, and pattern detection are each given distinct analytical contracts. This supports both implementation clarity and future extensibility.

E.6 ENDPOINT-LEVEL ENGINEERING SIGNIFICANCE

Endpoint-level documentation is important because it shows how high-level requirements are translated into API contracts. In a fintech context, this improves clarity for testing, review, frontend integration, and future expansion. It also shows that the platform is capable of being reasoned about as a professional system rather than only as a visual prototype.

CHAPTER F

EXTENDED DATA MODEL, SECURITY, AND REQUIREMENT TRACEABILITY

F.1 DATA MODEL DEEP DIVE

The data model of FinHQ is broad enough to support a meaningful traceability view. The ‘User’ entity anchors identity, security state, and ownership relationships. ‘PasswordHistory’ and ‘UserSession’ support secure lifecycle tracking. ‘Stock’, ‘OHLCVData’, and ‘FundamentalData’ support the market-data domain. ‘Portfolio’, ‘Position’, ‘Transaction’, ‘Order’, and ‘Trade’ support the action domain. ‘Alert’, ‘Notification’, and ‘AlertHistory’ support monitoring. ‘Strategy’ and ‘BacktestRun’ support simulation. ‘Post’, ‘Comment’, ‘Like’, and ‘Follow’ support engagement. Educational entities support structured learning. This breadth makes the data model one of the project’s strongest long-term assets.

F.2 SECURITY CONTROL REVIEW

Security is not a side note in this repository. The implemented controls include token-type distinction, Argon2-based password hashing with migration support, password-policy enforcement, breach screening, login lockouts, session recording, password-history rules, TOTP support, rate limiting, and protected dependency guards. A final report must make these controls explicit because they significantly improve the credibility of a finance-oriented academic project.

F.3 FUNCTIONAL REQUIREMENT TRACEABILITY

The platform requirements are traceable to implementation areas. User registration maps to auth routes, validation, password policies, and persistence. Stock search maps to stock routes and data services. Trading maps to broker and order logic. Alerts map to alert services and notification state. Educational requirements map to module and lesson entities. AI requirements map to the ML service surface. This traceability is important because it proves that the project is not only designed but also partially realized through concrete implementation boundaries.

F.4 NON-FUNCTIONAL TRACEABILITY

Non-functional concerns such as maintainability, modularity, security, and reproducibility are visible in the project structure itself. The use of service separation, containerization, typed schemas, route grouping, and protected dependencies demonstrates that these qualities are built into the codebase rather than treated as afterthoughts.

CHAPTER G

EXTENDED OPERATIONAL AND SUBMISSION NOTES

G.1 USER MANUAL PERSPECTIVE

From a user-manual perspective, the system can be understood in four phases: access, research, action, and review. Access begins with account creation and login. Research includes dashboard, technical analysis, fundamentals, news, and risk views. Action includes alerts, strategies, backtesting, and paper trading. Review includes portfolio summaries, notifications, and continued learning. This phased understanding makes the platform easier to explain to evaluators and new users.

G.2 SUBMISSION PERSPECTIVE

For a university submission, the system is valuable because it supports explanation across architecture, code structure, data modeling, security, and AI integration. The report can therefore be used not only as documentation but also as a viva guide, because each section maps cleanly to demonstrable implementation areas.

G.3 FINAL NOTE ON REPORT COMPLETENESS

The final report is intentionally broader than the draft and revised versions. It includes more architectural reasoning, more module-level discussion, more traceability, more security explanation, and more operational context. This is appropriate because the final report is expected to function as the most complete technical and academic record of the project.

CHAPTER H

BACKEND API INVENTORY

Endpoint	Responsibility
/api/v1/auth/register	Registers a user after uniqueness checks, password policy validation, breach screening, and password-history initialization.
/api/v1/auth/login/access-token	Authenticates users, enforces lockouts, supports 2FA branching, and issues access plus refresh tokens.
/api/v1/auth/login/verify-2fa	Completes a TOTP-based login challenge and opens a real session.
/api/v1/auth/refresh	Refreshes access tokens using refresh-token semantics.
/api/v1/auth/2fa/setup	Generates a TOTP secret and provisioning URI.
/api/v1/auth/2fa/verify	Finalizes 2FA setup.
/api/v1/users/me	Returns the authenticated user profile.
/api/v1/users/forgot-password	Generates a reset token with demo-mode response behavior.
/api/v1/users/reset-password	Validates reset token, enforces password rules, blocks reuse, and invalidates sessions.
/api/v1/stocks/search	Searches stored stocks and supplements results from the curated Indian stock universe.
/api/v1/stocks/market-movers	Returns gainers and losers using live or fallback data.
/api/v1/stocks/index-stocks	Supplies Nifty, Sensex, or combined symbol catalogs.
/api/v1/stocks/{ticker}	Returns or hydrates stock master information.
/api/v1/stocks/{ticker}/ohlcv	Provides candle data from DB or live provider fallback.
/api/v1/stocks/{ticker}/indicators	Calculates SMA, EMA, RSI, MACD, and Bollinger Bands server-side.
/api/v1/stocks/{ticker}/quote	Supplies latest quote data.
/api/v1/stocks/{ticker}/fundamentals	Returns fundamentals and valuation-oriented data.
/api/v1/stocks/{ticker}/news	Fetches and sentiment-enriches asset-level news.
/api/v1/trading/orders	Places or lists paper-trading orders.

/api/v1/trading/orders/{orderId} Cancels pending paper orders.
 /api/v1/trading/positions Returns holdings enriched with simulated live prices.
 /api/v1/trading/portfolio/summary Provides aggregate portfolio values.
 /api/v1/alerts Creates, lists, and deletes alerts.
 /api/v1/alerts/notifications Lists and marks notifications as read.
 /api/v1/strategies Lists, creates, and deletes strategy definitions.
 /api/v1/social/feed Returns social feed content.
 /api/v1/social/posts Creates community posts.
 /api/v1/social/posts/{id}/comments Adds comments to community posts.
 /api/v1/education/seed Seeds learning content.
 /api/v1/education/modules Lists modules and learning state.
 /api/v1/education/lessons/{id}/marks Marks lessons complete.

CHAPTER I

ML API INVENTORY

Endpoint	Responsibility
/api/v1/features/calculate	Generates technical features, candlestick patterns, and support-resistance levels.
/api/v1/training/train	Runs model-training related workflows.
/api/v1/sentiment/predict	Scores text sentiment for financial context.
/api/v1/prediction/train	Trains an LSTM-based price model.
/api/v1/prediction/forecast	Forecasts the next price from recent history.
/api/v1/recommendation/analyze	Produces a recommendation from price behavior and optional news text.
/api/v1/technical/analyze	Runs technical-analysis logic for chart-based insight.
/api/v1/risk/analyze	Calculates portfolio risk metrics from holdings and history.
/api/v1/fundamental/analyze	Computes DCF and health-score outputs.
/api/v1/backtest/run	Executes a simplified backtesting routine.
/api/v1/pattern-detection/detect-from-data	Runs ensemble pattern detection and optionally returns annotated charts.
/api/v1/pattern-detection/detect-image	Placeholder image-based path pending trained vision weights.
/api/v1/pattern-detection/patterns-history	Returns stored detection history.
/api/v1/pattern-detection/supported-patterns	Lists supported classical chart patterns.

CHAPTER J

DATA DICTIONARY AND ENTITY REGISTER

Entity	Purpose
User	Identity, account status, risk profile, verification state, lockouts, timestamps, and security relationships.
PasswordHistory	Tracks previous password hashes to block reuse.
UserSession	Stores per-device session tokens, expiry, fingerprint, and IP metadata.
Stock	Stores symbol identity and company metadata.
OHLCVData	Stores historical candle data.
FundamentalData	Stores company-level research and fundamentals.
Portfolio	Stores top-level simulated account state.
Position	Stores active holdings.
Transaction	Stores trading ledger entries.
Order	Stores paper-order state and order lifecycle.
Trade	Stores trade-related activity linked to execution.
Strategy	Stores strategy definitions and related settings.
BacktestRun	Stores simulation results.
Alert	Stores threshold-monitoring definitions.
Notification	Stores delivered alert or system notifications.
AlertHistory	Stores triggered alert events.
Post	Stores social or community posts.
Comment	Stores post-level interaction.
Like	Stores post engagement.
Follow	Stores user-to-user following relationship.
Module	Stores education module metadata.
Lesson	Stores lesson units.
UserProgress	Stores progress against educational content.
Enrollment	Stores learner-course relationship.
Quiz	Stores assessment items.
QuizSubmission	Stores user submission state.
Watchlist	Stores a user monitoring group.
WatchlistItem	Stores symbols inside watchlists.

MLPrediction	Stores predictive output context.
NewsArticle	Stores or represents article-level news data.
NewsSentiment	Stores enriched sentiment-oriented news interpretation.

CHAPTER K

SECURITY CONTROL REGISTER

Control	Significance
Token type claims	Distinguishes access and refresh semantics and reduces ambiguity in token handling.
Argon2 hashing	Improves password-storage security beyond basic unsalted or weak hashing strategies.
Password policy	Enforces stronger credentials during registration and reset flows.
Breached-password checks	Prevents known compromised passwords from being reused.
Lockout behavior	Reduces brute-force risk after repeated invalid login attempts.
Session metadata recording	Improves auditability and future session management possibilities.
Password history enforcement	Reduces the risk of cycling between previously used passwords.
TOTP support	Adds second-factor account protection.
Rate limiting	Protects sensitive auth endpoints from rapid abuse.
Protected dependencies	Ensures user-scoped APIs require valid authentication state.

CHAPTER L

USE CASE AND REQUIREMENT TRACEABILITY MATRIX

Use Case	Primary Module	Traceability Notes
Register user	Auth backend + front-end	Traceable to registration route, password rules, breach checks, and user model.
Login user	Auth backend + front-end	Traceable to login route, logout logic, token generation, and session recording.
Reset password	Users backend + auth screens	Traceable to reset endpoints, password history, and password-strength UI.
Search stock	Stock service + technical UI	Traceable to stock search API, curated Indian stock catalog, and ticker search component.
View chart data	Stocks API + technical UI	Traceable to OHLCV route, chart component, and indicator API.
Read fundamentals	Stocks API + fundamental UI	Traceable to fundamental data route and company analysis cards.
Read news	Stocks API + news UI	Traceable to news route and sentiment badge rendering.
Place paper order	Trading API + trade UI	Traceable to broker service, order routes, and trade form.
Create alert	Alerts API + alerts UI	Traceable to alert creation route, notification state, and trigger history.
Run backtest	ML API + backtest UI	Traceable to backtest service and strategy evaluation screen.
Inspect risk	ML API + risk UI	Traceable to risk service and risk-interpretation displays.
Use learning module	Education API + learn UI	Traceable to education routes and progress entities.
Post to community	Social API + community UI	Traceable to post and comment routes plus social models.

CHAPTER M

FORMAL TEST CASE MATRIX

Test ID	Scenario	Test Condition	Expected Result
TC-01	Registration	Valid user details with strong password	Account is created and user can proceed to login.
TC-02	Registration	Email already exists	System rejects request with a clear uniqueness message.
TC-03	Registration	Weak password	System rejects request and shows password-policy failure.
TC-04	Login	Valid credentials without 2FA	Access and refresh tokens are issued and session opens.
TC-05	Login	Invalid password	Login fails and failed-attempt counter increments.
TC-06	Login	Repeated invalid password attempts	Temporary lockout is enforced.
TC-07	2FA verification	Correct TOTP code	Login challenge completes successfully.
TC-08	2FA verification	Invalid TOTP code	Request is rejected and user remains unauthenticated.
TC-09	Forgot password	Registered email address	Reset token flow is generated according to system mode.
TC-10	Reset password	Previously used password	Reset is rejected due to password-history enforcement.

TC-11	Stock search	Known Indian ticker	Matching symbol list is returned.
TC-12	OHLCV retrieval	Valid ticker and time-frame	Candle data is returned in chronological structure.
TC-13	Indicator generation	Valid ticker and period	SMA, EMA, RSI, MACD, and Bollinger results are produced.
TC-14	Fundamentals lookup	Valid symbol	Fundamental metrics and analysis payload are returned.
TC-15	News retrieval	Valid symbol	Asset news items are returned with sentiment context where available.
TC-16	Paper buy order	Enough cash balance	Order is accepted and positions plus cash update correctly.
TC-17	Paper buy order	Insufficient cash balance	Order is rejected without corrupting portfolio state.
TC-18	Paper sell order	Quantity owned	Sell order is accepted and holdings are reduced appropriately.
TC-19	Paper sell order	Quantity exceeds holdings	System blocks sale and preserves position integrity.
TC-20	Portfolio summary	Authenticated user with activity	Aggregate market value, cash, and P&L are returned.
TC-21	Alert creation	Valid ticker and threshold	Alert record is created successfully.
TC-22	Alert notification read	Existing unread notification	Notification state changes to read.
TC-23	Strategy creation	Valid strategy payload	Strategy record is persisted and listed for user.

TC-24	Backtest execution	Valid strategy parameters	Simulation returns performance-oriented output.
TC-25	Risk analysis	Valid holdings and history	Risk metrics such as VaR and drawdown are returned.
TC-26	Sentiment analysis	Financial news text sample	Sentiment label and confidence output are produced.
TC-27	Pattern detection	Valid candle sequence	Supported pattern set or no-pattern result is returned.
TC-28	Education completion	Valid lesson completion request	User progress updates successfully.
TC-29	Community post creation	Authenticated user with valid text	Post is created and becomes visible in feed.
TC-30	WebSocket subscription	Valid channel subscription	Client receives simulated updates for subscribed topic.

CHAPTER N

MODULE-TO-SCREEN AND SERVICE MAPPING

Business Module	Frontend faces	Sur-	Primary Services	Documentation Value
Authentication and identity	/login, /register, /forgot-password, /reset-password, /2fa-setup		Auth routes, user routes, session handling, TOTP logic	Demonstrates secure onboarding and account protection.
Dashboard intelligence	/dashboard		Portfolio aggregation, alert retrieval, market movers, summary composition	Shows how multiple services are combined into one executive view.
Portfolio monitoring	/portfolio		Broker service, quote enrichment, portfolio summary APIs	Connects account state with current market context.
Technical analysis	/technical		Stock data, indicator generation, pattern detection, chart data APIs	Demonstrates core analytical workflow for investor research.
Fundamental analysis	/fundamental		Fundamental retrieval, valuation logic, health-score logic	Documents research-oriented data transformation.
News intelligence	/news		News retrieval and sentiment enrichment	Shows event-driven research support.
Trading simulation	/trade		Broker service, orders, positions, portfolio summary	Explains transactional and ledger-oriented backend behavior.

Strategies and backtesting	/strategies, /backtest	Strategy backtest ML routing	CRUD, service, ML routing	Shows quantitative experimentation and simulation support.
Risk analytics	/risk	Risk portfolio-history ingestion, interpretation output	Risk service, portfolio-history ingestion, interpretation output	Demonstrates portfolio-level analytics and decision support.
Alerts and notifications	/alerts	Alert service, notification state, alert history	Alert service, notification state, alert history	Documents event threshold management and user reminders.
Community interaction	/community	Social feed, posts, comments	Social feed, posts, comments	Shows collaborative and engagement-oriented feature coverage.
Learning and education	/learn	Education module service, lessons, progress tracking	Education module service, lessons, progress tracking	Demonstrates non-transactional instructional support.

CHAPTER O

IMPLEMENTATION MILESTONE SUMMARY

Phase	Major Implementation Focus	Outcome
Phase 1	Project setup, route planning, backend scaffolding, database configuration, and container setup	Established runnable foundation for frontend, backend, ML service, database, Redis, and supporting infrastructure.
Phase 2	Authentication workflows, user registration, login, password policy, session handling, and reset support	Created a security-oriented access layer suitable for fintech-style platform usage.
Phase 3	Stock search, quotes, OHLCV ingestion, indicator computation, and chart-based technical analysis	Enabled core market-data workflows and symbol-level research functionality.
Phase 4	Fundamental analysis, news integration, sentiment enrichment, and dashboard assembly	Expanded research capabilities beyond raw prices into qualitative and valuation-oriented insight.
Phase 5	Paper trading, positions, transactions, portfolio summary, and order-state management	Added simulation-oriented execution and portfolio lifecycle behavior.
Phase 6	Alerts, notifications, community, and education modules	Improved platform breadth and user engagement across informational and social workflows.
Phase 7	ML endpoints for prediction, recommendation, risk, back-testing, sentiment, and pattern detection	Added analytics depth and established a clear AI narrative supported by dedicated service boundaries.

Phase 8	Documentation consolidation, report refinement, module analysis, test planning, and final report preparation	Produced an academic-grade explanation of architecture, implementation, results, and future scope.
---------	--	--

CHAPTER P

OPERATIONAL RISK AND MITIGATION REGISTER

Risk Area	Nature of Risk	Potential Impact	Mitigation	Direction
Live market data dependence	External providers may rate-limit or change response structures	Incomplete or delayed quotes, candles, or re-search data	Introduce abstraction, caching, and fallback data sources.	provider retries,
Mock-data screens	Some frontend views still use representative data	User may assume full live integration where it is not yet complete	Clearly label staged modules and progressively replace mocks with authenticated APIs.	
Token storage model	Local token storage may be less robust than hardened cookie-based patterns	Increased exposure under browser-compromise scenarios	Evaluate secure cookie strategies and stricter session-hardening approaches.	
Model performance drift	ML outputs may degrade when market conditions differ from training assumptions	Lower trust in forecasting, risk scoring, or recommendation quality	Add experiment tracking, validation datasets, retraining schedules, and monitoring.	
Testing depth	Automated test coverage is limited	Undetected regressions may occur during refactoring	Introduce unit, integration, contract, and end-to-end testing in CI.	

Infrastructure portability	Local workflow not match production assumptions	Docker may directly production hosting	Deployment inconsistency across environments	Define environment-specific configs and cloud deployment playbooks.
Security misconfiguration	Secrets, algorithm settings, or rate-limit settings may drift across environments	CORS, set-tings, or rate-limit settings	Increased attack surface or authentication inconsistency	Add environment validation, secrets governance, and deployment checklists.
Vision-model dependency	Pattern image detection depends on trained model assets	Pattern image de-tection depends on trained model assets	Some image pathways remain incomplete or placeholder-oriented	Keep API behavior explicit and progressively add production-ready weights.

REFERENCES

- [1] E, Abhijith. “fintechphase2.” GitHub. Accessed 25 Mar. 2026. <https://github.com/Abhijith-E/fintechphase2>.
- [2] Vercel. “Next.js Documentation.” Next.js. Accessed 25 Mar. 2026. <https://nextjs.org/docs>.
- [3] Meta Open Source. “React Documentation.” React. Accessed 25 Mar. 2026. <https://react.dev/>.
- [4] Ramírez, Sebastián. “FastAPI Documentation.” FastAPI. Accessed 25 Mar. 2026. <https://fastapi.tiangolo.com/>.
- [5] Docker Inc. “Docker Documentation.” Docker. Accessed 25 Mar. 2026. <https://docs.docker.com/>.
- [6] PostgreSQL Global Development Group. “PostgreSQL Documentation.” PostgreSQL. Accessed 25 Mar. 2026. <https://www.postgresql.org/docs/>.
- [7] PyTorch Contributors. “PyTorch Documentation.” PyTorch. Accessed 25 Mar. 2026. <https://docs.pytorch.org/docs/stable/index.html>.
- [8] MLflow Contributors. “MLflow Documentation.” MLflow. Accessed 25 Mar. 2026. <https://mlflow.org/docs/latest/index.html>.